SOLAT

Big data

Livre blanc

SOAT en quelques mots

Fondé par Michel Azria et David-Eric Levy, SOAT est un cabinet de conseil IT qui fait grandir depuis 16 ans une communauté de consultants spécialisés dans le développement et l'optimisation de systèmes d'information.

Au quotidien, ce sont plus de 360 collaborateurs de talent qui accompagnent nos clients sur des problématiques technologiques et organisationnelles complexes : développement d'applications spécifiques, performance et qualité du code, déploiement continu, choix technologiques, conception d'architectures IT, transformation agile et modernisation du système d'information.

Pour garantir à nos clients des prestations de haute qualité et une expertise toujours à la pointe, nous avons placé l'innovation technologique et la capitalisation de nos savoir-faire au cœur de notre stratégie.

Conférences, livres blancs, avis d'expert, blog, communautés techniques et de pratiques… Nous favorisons le partage des connaissances pour faire continuellement progresser nos consultants et apporter toujours plus de valeur ajoutée à nos clients. Cette démarche de partage et de capitalisation ancrée dans les gènes de SOAT participe à la diffusion des savoirs.

Allier savoir-faire et savoir-être reste notre défi au quotidien pour rendre notre organisation toujours plus performante et épanouissante.

ÉDITO

En 2017, ce sont au moins 100 projets Big Data qui seront concrètement mis en œuvre en France.

Le travail d'évangélisation mené sur le nouveau paradigme des données a payé : il a posé les bases pour que les organisations puissent en faire un usage qui soit cohérent avec la nature de leur activité. Au-delà de la théorie, nous entrons donc vraiment au cœur du sujet : la mise en production de projets Big Data, c'est maintenant.

Des décideurs et des experts Big Data

En l'occurrence, deux sujets doivent s'enchaîner harmonieusement dans les organisations pour pleinement profiter de cette concrétisation. Maintenant que la volonté des directions générales est là, il est urgent de faire émerger des convictions business explicites - pour ceux qui n'ont pas encore franchi cette étape. Puis, il faut être capable de traduire techniquement et opérationnellement ces convictions.

« AU-DELÀ DE LA THÉORIE, NOUS ENTRONS DONC VRAIMENT AU CŒUR DU SUJET : LA MISE EN PRODUCTION DE PROJETS BIG DATA, C'EST MAINTENANT. »

C'est pourquoi, ce livre blanc s'attache à la fois à donner des clés aux décideurs pour affiner leur stratégie Big Data, et à leurs équipes, pour opérer concrètement un tel projet dans toute sa complexité technologique.

Pour les premiers, il s'agit d'aller au-delà de la « nébuleuse » du terme Big Data et d'identifier les cas d'usages, les réalités tangibles, qui leur permettront de traduire en une vision business spécifique les possibilités offertes. Il est également important qu'ils puissent se forger une vision des rôles et des responsabilités nécessaires pour mener à bien ces projets, afin de s'entourer des bonnes personnes. On ne le répètera jamais assez : le Big Data demande d'avoir les idées claires sur ce que l'on veut en faire. Et plus encore qu'aux experts du sujet, cette clarté dans la réflexion doit être l'apanage des dirigeants, dont le business va être profondément modifié par de tels projets.

Pour leurs équipes, ce livre blanc est l'occasion de voir décrit en profondeur les enjeux techniques – de stockage, de traitement des données ou encore de restitution – qui permettront de pérenniser l'approche technologique de l'entreprise... • • •

• • •

L'univers du Big Data est en effet peuplé de solutions technologiques complexes, qui ont souvent des durées de vie courtes. Les combinaisons possibles en fonction des cas d'usage sont donc nombreuses et changent régulièrement. Les infrastructures qui les soutiennent sont elles-mêmes variées, ce qui implique de faire des choix engageant en connaissance de cause. Il est loin le temps où l'on pouvait espérer résumer ces enjeux à NoSQL… Aujourd'hui, l'explosion technologique généralisée rend centraux les enjeux d'adaptation, de maîtrise des coûts, de conservation de la valeur acquise. Bref, il est nécessaire de dépasser « l'expérimentation one shot » qui avait pu prévaloir, pour assumer une industrialisation éprouvée.

« UNE PLATEFORME BIG DATA EST UN ORGANISME VIVANT DONT IL FAUT S'OCCUPER SUR LE LONG TERME. »

Réconcilier vision stratégique et technique

Une plateforme Big Data est comme un organisme vivant dont il faut s'occuper avec attention sur le long terme. Certains choix que fera l'entreprise peuvent exclure de nombreux cas d'usage auxquels elle n'avait pas pensé à l'origine. Et contrairement à ce que certains acteurs ont pu estimer : une plateforme neutre, générique, qui pourrait répondre à tous les cas de figure… est tout simplement illusoire.

Il est donc bien nécessaire de faire émerger chez eux également des convictions, à partir de leurs connaissances des volumes et de la nature des données concernées ou également des temps de restitution requis…

Vision stratégique des dirigeants et vision technique de leurs équipes sont bien les deux faces de la même pièce ; il est plus important que jamais de pouvoir les réconcilier et les traiter tour à tour harmonieusement.

Nous espérons que ce livre blanc répondra aux questions que vous pouvez avoir sur un sujet qui n'est pas un simple buzzword, et qui vous permettra de faire de 2017 l'année du Big Data concrètement mis en œuvre dans votre entreprise.

En vous en souhaitant une agréable lecture.

Grégory Boissinot – CTO de SOAT

TABLE DES MATIÈRES

P.06 > PARTIE I : BIG DATA : QUELS USAGES ET PRÉREQUIS ?

P.08 > 1 Le Big Data, c'est quoi ?

P.11 > 2 Pourquoi se lancer dans un projet Big Data ?
P.12_Qui doit s'intéresser au Big Data ?
P.13_Comment identifier les sujets stratégiques à traiter ?
P.15_De quels cas d'utilisation puis-je m'inspirer ?
P.19_Les différents niveaux du modèle de maturité

P.20 > 3 Quels sont les coûts d'un projet bien mené ?
P.21_Le coût de qualité de la donnée
P.21_Les coûts d'infrastructure
P.22_Les coûts de licence
P.22_Les coûts d'apprentissage
P.23_Prérequis de fédération et de gouvernance des données

P.24 > 4 Le Cloud est-il un prérequis ?
P.25_Des principes adaptés aux besoins Big Data
P.26_Des conséquences à anticiper

P.27 > 5 Sur quelle Dream Team compter pour réussir mon projet ?
P.28_Les joueurs clés du match
P.29_Les autres atouts de l'équipe

P.30 > 6 Que changer dans mon entreprise pour m'assurer de créer de la valeur avec le Big Data ?
P.31_Des valeurs et des méthodes
P.32_Une nouvelle réflexion sur les KPI
P.33_Une adaptation aux risques et contraintes du Big Data

P.35 > Conclusion Partie I

P.36_PARTIE II : BIG DATA : QUEL ÉCOSYSTÈME TECHNOLOGIQUE ?

P.38 > 1 Acquisition / Extraction
P.39_L'intégration de données via un système de fichiers
P.39_Acquisition via un Data Lake
P.40_Extraction des données via les ETL
P.40_L'émergence de la solution Apache Kafka

P.41 > 2 Stockage
P.42_Les bases de données relationnelles
P.44_Écosystème des bases de données NoSQL
P.45_Gestion de la temporalité
P.48_Système de fichiers distribués
P.48_Processus applicatif distribué
P.49_Recherche distribuée
P.49_Configuration distribuée

P.50 > 3 Flux continu d'événements
P.51_Fonctionnement d'un flux continu d'événements
P.51_Événements bruts versus Événements agrégés : des cas d'utilisation différents

P.54 > 4 Intégration des données et traitement
P.55_Problématiques d'intégration des données
P.56_Techniques de mise à jour de plusieurs sources de données
P.57_Change Data Capture (CDC)
P.58_Notion de vues
P.58_Philosophie Unix
P.59_Kafka, un outil central
P.60_Traitement des données
P.60_Traitement en mode Batch
P.61_Traitement en mode Micro-Batch
P.61_Traitement en mode temps-réel

P.62 > 5 Analyse, Restitution et Visualisation
P.63_Valorisation des données
P.63_Restitution

P.64 > 6 Architectures des systèmes Big Data
P.65_Des propriétés communes
P.66_Architecture Hadoop
P.67_Architecture dite "Batch incrémental"
P.67_Architecture Lambda
P.68_Les architectures émergentes à base d'un Stream d'événements
P.70_Architecture Kappa
P.70_Architecture SMACK

P.71 > 7 Infrastructure Big Data
P.72_Sécurité
P.72_Cloud
P.72_Dimensionnement
P.73_Supervision

P.75 > Conclusion Partie II

P.76 > Vision et prospective

P.79 > Auteurs

Quels usages et prérequis ?

Développer une vision actionnable du Big Data est une fine alchimie. Le terme, largement utilisé, et souvent dans des contextes très différents, camoufle sous une seule étiquette des réalités variées et de très nombreux ingrédients nécessaires à son existence concrète dans l'entreprise. Loin d'être une baguette magique, une approche Big Data réussie est un processus savamment réfléchi, qui devra au préalable répondre à de nombreuses questions avant même d'entrer dans le détail des réalisations techniques par la suite nécessaires.

Les pages qui suivent abordent ces questions initiales, devant être absolument prises en compte par les décideurs. Elles permettent de clarifier le sens que l'entreprise va conférer à « son » Big Data.

P.08 > 1 Le Big Data, c'est quoi ?

P.11 > 2 Pourquoi se lancer dans un projet Big Data ?

P.20 > 3 Quels sont les coûts d'un projet bien mené ?

P.24 > 4 Le Cloud est-il un prérequis ?

P.27 > 5 Sur quelle Dream Team compter pour réussir mon projet ?

P.30 > 6 Que changer dans mon entreprise pour m'assurer de créer de la valeur avec le Big Data ?

P.35 > Conclusion Partie I

Le Big Data, c'est quoi ?

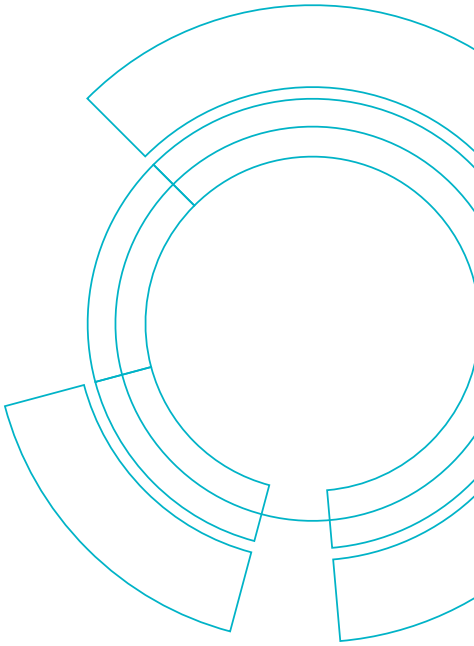
Le Big Data est une discipline nouvelle qui se situe au croisement de plusieurs domaines :

les statistiques, la technologie,
les bases de données et les métiers (marketing, RH, commercial, etc.).

Si cette discipline en tant que telle est nouvelle, le concept « Big Data » permettant de stocker un nombre indicible d'informations sur une base numérique, est lui omniprésent depuis plusieurs années.
Le grand public l'appréhende et l'expérimente au quotidien à travers de multiples scénarios : l'usage du web, des messageries comme Gmail, des réseaux sociaux à travers lesquels se dégagent certains processus comportementaux, ou encore, dans l'usage des grands volumes de données produits par les puces RFID (Radio Frequency IDentification) utilisées dans les chaînes d'approvisionnement de marchandises.

Tous ces scénarios font partie de la vie quotidienne, et ne datent pas d'hier. Les codes-barres créés dans les années 70 ont toujours produit un immense volume de données. Aussi, là où le Big Data est véritablement créateur de valeur, ce n'est pas dans la production massive de données, mais dans le fait de chercher à les conserver, à les croiser et à les analyser. Tout l'enjeu du Big Data est en effet de s'approprier la donnée pour mieux la comprendre et l'exploiter.

Si aucune définition universelle ne peut être donnée à cet objet complexe et polymorphe qu'est le Big Data, l'image des « 5 V » est communément reprise et admise pour l'exprimer. Il s'agit notamment d'un **Volume** de données considérable à traiter, une grande **Variété** d'informations (venant de diverses sources, non-structurées, organisées, etc.) et un certain niveau de **Vélocité** à atteindre, correspondant à la fréquence de création, de collecte et de partage des données. Les deux autres V concernent la **Véracité** de l'information et la **Valeur** (autrement dit le profit) que l'on peut en tirer.



LES 5 V

Volume
Le volume de données produites par les entreprises est en constante augmentation. Aujourd'hui, les données se comptent en téraoctets ou pétaoctets. Mais un projet Big Data peut également faire sens sur un volume plus réduit. Ce qui compte ici c'est la notion de scalabilité et d'architecture « à l'échelle » qui permettra ensuite de l'adapter selon les nouveaux besoins.

Variété
Le but du Big Data est bien de pouvoir collecter des données quelles que soient la source et la forme qu'elles prennent. Donner du sens à la diversité, c'est provoquer un effet Médicis[1] dans votre architecture.

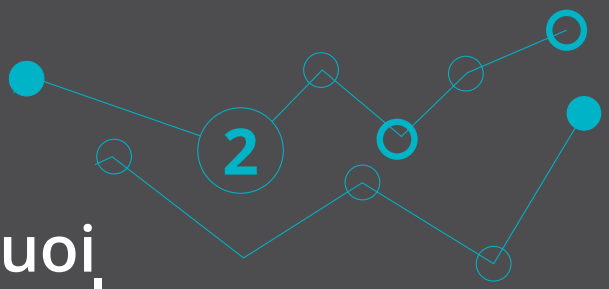
Vélocité
De nombreux types de données voient la valeur de leur utilisation décroître avec le temps. Heureusement, utiliser les données à mesure qu'elles sont collectées est désormais possible grâce au Big Data. Mais qui dit variété, dit gestion de cette variété : puissance de calcul, outils d'analyse performants, sur mesure et évolutivité, sont nécessaires pour garantir leur traitement rapide. Pour cela une architecture adaptée est requise, ainsi qu'une véritable politique d'historisation, car l'accumulation de données jamais exploitées, est un risque.

ET AUSSI...

Visibilité ou Valeur
Pour bien utiliser la donnée, il est nécessaire de pouvoir y accéder de manière compréhensible rapidement. L'architecture et la façon dont l'accès et la visualisation sont prévues, doivent favoriser l'émergence du « sens ». Dans un contexte d'infobésité, il s'agit d'être capable de se concentrer sur les données ayant une réelle valeur et étant actionnables.

Véracité
Evoquer la véracité, c'est pointer le problème de la qualité des données. Traiter de la variété de façon fiable nécessite des règles et la qualité qui en découlera validera à la fois le sens et la valeur que l'entreprise entend créer avec le Big Data.

[1] > Effet Medicis : concept développé par l'entrepreneur américano-suédois Frans Johansson pour expliquer comment reproduire en entreprise, l'explosion de créativité qui a eu lieu à la Renaissance autour de la ville de Florence.



Pourquoi se lancer dans un projet Big Data ?

Sans surprise, l'erreur fondamentale qui risque de compromettre tout projet Big Data, revient à se lancer sans but bien défini. Et soyons clair : vouloir imiter ou devancer ses concurrents n'est pas en soi une raison suffisante pour faire du Big Data.

A l'inverse, diminuer les risques de fraude, améliorer la prise de décision, développer une meilleure compréhension de ses clients ou proposer un nouveau service créateur de valeur sont d'excellentes raisons d'aller explorer les possibilités offertes par les usages et les technologies Big Data.

Ainsi, c'est en identifiant un besoin métier concret, hautement stratégique pour le développement de son activité, qu'il deviendra intéressant de se consacrer à un tel projet.

Reste donc à déterminer qui sont les acteurs dans l'entreprise les plus à même d'identifier ce besoin stratégique et comment ils peuvent s'assurer que le Big Data y apporte les réponses adaptées.

Qui doit s'intéresser au Big Data ?

« La seule vraie façon de rater son projet Big Data est de ne pas essayer »

Denis Oblin, spécialiste du Big Data chez Memorandum et membre du comité scientifique du Data Intelligence Forum 2016

De manière assez évidente, les services marketing, client ou produit trouveront avec le Big Data une extraordinaire opportunité de créer de la valeur métier, en rendant accessibles des données qui ne l'étaient pas, pour les analyser et ainsi optimiser leur CRM, leur marketing web ou mobile, leur stratégie d'achat d'espace ou d'expérience en magasin.

Les commerciaux, les assurantiels, les industriels et les professionnels de l'énergie trouveront également un intérêt évident à l'analyse tendancielle et la gestion des risques que le Big Data permet.

Au-delà de la sphère strictement économique et privée, d'autres domaines professionnels peuvent en tirer de la valeur comme les médecins pour répondre à des problématiques épidémiologiques, les météorologues pour anticiper les changements climatiques, les politiques dans le cadre de campagnes électorales, etc.

Vous l'aurez compris, presque tout le monde peut trouver dans le Big Data un potentiel de progrès majeur. Ce point de départ fait sortir tout projet Big Data du seul domaine des spécialistes techniques, IT et, évidemment, de la « data ».

Puisqu'il est crucial de déterminer un besoin hautement stratégique en amont pour s'assurer de la réussite des usages Big Data dans l'entreprise, ceux-ci doivent avant tout trouver l'oreille des dirigeants afin de s'intégrer dans leur vision transversale de l'entreprise.

La mise en musique de cette vision sera ensuite le fait de chefs d'orchestre ayant une compréhension des technologies et de leurs enjeux autour de la donnée et du système d'information lui-même.

Comment identifier les sujets stratégiques à traiter ?

Un assureur français dispose d'une vision complète de son client, indépendamment des produits auxquels il a souscrit. Cette société d'assurance utilise le Big Data pour prédire les effets de la loi Hamon qui facilite une rupture de contrat avec son assureur, en se dotant d'outils mesurant les impacts d'un changement législatif.

Une banque française sait quels sont ses produits financiers les plus vendus et sur lesquels sa marge est la meilleure. Cette banque utilise le Big Data pour réaliser une détection fine des fraudes, avec la prise en compte de signaux faibles.

Un comparateur d'assurances a mis en place une solution Big Data pour anticiper le moment où son SGBDR[1] ne serait plus capable de faire face au volume croissant de requêtes et données.

Confronté à des lenteurs de calcul générées par la taille de ses bases (plus de 2,5 milliards de lignes), le site Mappy a conçu une application Big Data en interne qui, en court-circuitant les technologies du marché, a divisé par mille le temps de calcul.

Ces quatre exemples ont en commun d'inscrire l'initiative Big Data de l'entreprise au cœur de son activité, afin de fournir un avantage concurrentiel et un levier très facilement compréhensible non seulement pour tous les collaborateurs du métier, mais également pour des acteurs extérieurs comme les clients et les partenaires.

Pour identifier les sujets stratégiques qui permettront d'y parvenir, deux points clés : faire simple et itérer avec les métiers. En effet, la capacité d'une entreprise à gérer la transversalité et la complexité d'un tel projet n'est pas infinie. Il est donc vital de faire des choix simples mais structurants autant pour les aspects métier et techniques que d'un point de vue organisationnel et relationnel.

S'entêter à vouloir trouver « la meilleure solution possible » dès le démarrage du projet est utopique et risque de laisser une trace indélébile dans votre budget. D'autant que la complexité technique ne doit surtout pas prendre le pas sur une compréhension fine du métier.

Une démarche d'exploitation des données est par essence itérative. C'est d'autant plus vrai dans une optique Big Data. Une première modélisation des données va probablement lever, côté métier, de nouvelles interrogations conduisant à de nouvelles actions qui elles-mêmes, permettront une analyse plus fine des données.

De la même manière, votre compréhension technique va évoluer et s'améliorer au cours du temps. Elle permettra l'ajout de nouveaux outils et solutions au sein de votre SI, ouvrant des perspectives nouvelles pour l'exploitation des données. Il est d'ailleurs préférable d'itérer le plus possible sur la partie technique pour avoir suffisamment de temps à consacrer aux besoins métier, et ceci dès le début du projet. • • •

[1] > *SGBDR : Système de Gestion de Base de Données Relationnelle.*

Par quoi commencer alors ? Concentrez-vous sur les quatre briques de base du Big Data : stockage, analyse, présentation, supervision.

En effet, le Big Data a pour but d'exploiter des volumes de données en croissance exponentielle, difficiles à traiter avec des outils de gestion classiques comme les bases de données relationnelles. L'autre enjeu de taille est de traiter ces données le plus rapidement possible.

Pour gérer de manière optimale le cycle de vie des données, leur qualité, leur sécurité et leur traitement, les plateformes Big Data doivent mettre à disposition des technologies avancées capables de gérer les différents aspects liés au stockage des données, à leur analyse, à leur restitution et leur supervision.

ET SI ON FAISAIT UN POC ?

Les architectures Big Data ont aujourd'hui atteint un niveau de maturité suffisant pour que le concept ait largement été éprouvé. Il est pourtant tentant, afin de démontrer la faisabilité du projet, de commencer par la réalisation d'une preuve de votre concept.

Cependant, un Proof Of Concept (POC) a un coût non négligeable et peut être rapidement abandonné s'il ne correspond pas aux attentes du métier ou aux contraintes techniques.

L'ensemble des acteurs de la prestation sont d'accord pour dire que la durée moyenne d'un POC en Big Data se situe entre 2 et 6 mois et que son enveloppe budgétaire peut aller de 40K à 250K€ pour les plus ambitieux. A cela s'ajoute les « Big Data tour » (campagnes d'évangélisation Big Data au sein de l'entreprise) qui prennent du temps et de l'argent pour de l'avant-vente interne.

Le POC peut également générer de la frustration. En effet, l'aspect démonstratif se substitue à la mise en production de votre projet, alors qu'une architecture Big Data est faite pour évoluer dans le temps. Chacune de ses évolutions est déjà en soi une preuve d'un nouveau concept ou la réponse à un nouveau besoin.

Si vous souhaitez faire du Big Data, notre meilleur conseil, c'est de vous lancer. Toute la valeur d'une architecture Big Data repose sur les données « temps réel », alors que le POC vous oblige à repousser le moment de se mettre en conditions réelles et d'exploiter les « vraies données ». Un projet Big Data se construit de manière itérative et l'apprentissage se fait pour et par la donnée.

De quels cas d'utilisation puis-je m'inspirer ?

Comment les entreprises utilisent-elles le Big Data pour créer des projets clés ? Il suffit bien souvent de regarder au sein de son écosystème pour trouver de nombreuses typologies de projets qui pourront orienter l'entreprise dans sa recherche de valeur.

Les projets analytiques

Ils permettent de mettre en évidence certaines métriques sur les données afin d'améliorer les performances de vos produits. Ils sont notamment utilisés par les sites d'e-commerce pour connaître leurs statistiques de fréquentation par tranche horaire ou par zone géographique, leurs produits les plus consultés, etc.

Les prédictions métier

En faisant appel au Machine Learning, les entreprises peuvent désormais déduire de certaines données, les comportements futurs de leurs clients. C'est le cas, par exemple, des assureurs qui prévoient les demandes de résiliation des contrats d'assurance ou des sites de comparaison de voyages qui prédisent la tendance des tarifs d'un vol, sur les prochains jours.

La classification de données

De nombreux algorithmes de classification permettent d'identifier certaines catégories ou comportements. Les exemples les plus courants d'utilisation sont la détection de fraudes et de spams. Mais ils peuvent également être utilisés pour déduire un type d'activité physique effectuée (marche, course à pied, etc.) en fonction des mouvements enregistrés depuis un bracelet d'activité.

Les recommandations avancées

L'analyse des données et des comportements utilisateurs peut également être utilisée à des fins de recommandations client, notamment grâce au Machine Learning. Les sites d'e-commerce utilisent souvent ce type d'algorithme pour proposer des produits en fonction des articles consultés ou achetés précédemment. Tous les acteurs de VOD font désormais des suggestions de programmes en fonction de ce que leurs clients ont précédemment regardé. Tout comme les opérateurs téléphoniques qui peuvent analyser la consommation de leurs clients afin de leur faire des propositions de forfaits mieux adaptés.

MACHINE LEARNING

Le Machine Learning ou apprentissage automatique, est un domaine apparenté à l'intelligence artificielle, défini par la capacité d'apprentissage des machines. Son application au Big Data offre de très grandes opportunités pour les entreprises.

Il consiste dans la mise en place d'algorithmes dans le but d'obtenir une analyse prédictive à partir de données. L'objectif étant, par exemple, de prédire des comportements futurs des utilisateurs ou déduire leur état présent pour leur proposer des solutions plus pertinentes et la meilleure expérience possible. Avec le Machine Learning, on cherche avant tout à établir des liens de corrélation entre deux événements, sans pour autant qu'il y ait un lien de causalité.

Comment une machine peut-elle apprendre ?

Il existe trois principaux types d'apprentissages : l'apprentissage supervisé, l'apprentissage non supervisé et l'apprentissage par renforcement.

Le premier, l'apprentissage supervisé, consiste à baser l'apprentissage sur un ensemble de données dont on connaît le résultat de la prédiction. L'idée est d'utiliser ces données en entrée pour valider une fonction de prédiction permettant d'obtenir les bons résultats. Une fois validé, l'algorithme sera appliqué à un autre ensemble de données permettant d'en obtenir des prédictions. Il sera ainsi possible à l'avenir de prédire le marché immobilier d'une ville à court terme, en ayant comme données d'apprentissage les informations des ventes des derniers mois.

L'apprentissage non supervisé repose sur des données qui ne sont pas étiquetées. Il faut alors valider une hypothèse permettant de trouver une corrélation entre les données. Cet apprentissage n'est pas seulement utilisé lorsque l'on dispose de peu d'informations sur les données. Il peut également être utilisé pour savoir s'il y a une corrélation entre certaines informations.

L'apprentissage par renforcement est légèrement différent des deux premiers car le programme n'aura pas forcément d'indication réelle sur l'objectif qu'il doit atteindre. Il interagit dans un environnement et apprend à partir d'expériences. Il prend des décisions en fonction de cet environnement, ces décisions influencent son environnement qui réagira et lui permettra de savoir si sa stratégie est bonne ou non. Ce type d'apprentissage est particulièrement plébiscité dès lors que le nombre de règles à développer devient trop grand ou qu'elles sont trop complexes. Ce qui est notamment le cas pour les voitures autonomes.

Tous ces modèles nécessitent un ensemble de données d'apprentissage (ou un environnement) représentatif et de qualité. Reste que cet apprentissage est également à superviser. Une hypothèse validée à un instant T ne sera peut-être plus valable 6 mois plus tard. Il faudra contrôler régulièrement les résultats et les réajuster le cas échéant, au besoin.

La simulation d'un marché ou d'un environnement

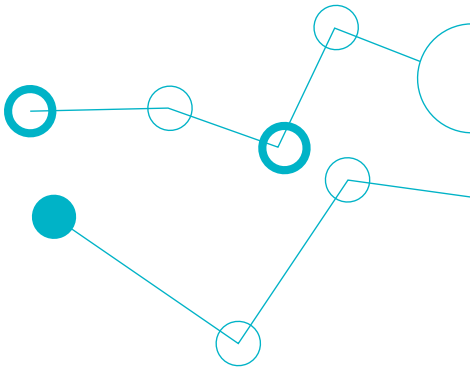
L'abondance d'informations et notre capacité à les traiter nous permet également de simuler une multitude de « situations type ». Pour optimiser une chaîne de production par exemple, il est nécessaire de connaître le comportement d'un produit dans certaines situations (comme un crash test), simuler son cycle de vie, tester virtuellement ses matériaux, avant de lancer les tests réels ou la production du produit. Dans le secteur de la banque-finance, il est même possible de recréer tout un écosystème de marchés économiques et de simuler un ou plusieurs événements, pour connaître les risques et les impacts qu'ils pourraient entraîner.

L'amélioration de l'expérience client

Les utilisateurs entrent en contact avec votre marque par de multiples biais. Etre en mesure de suivre le parcours d'un client à toutes les étapes de son processus d'achat pour mieux comprendre ses comportements et ses habitudes de consommation, est un défi auquel répond le Big Data. L'objectif sous-jacent étant d'adapter le mode de communication à l'utilisateur, d'améliorer les processus de vente, d'optimiser les investissements publicitaires, le service client et évidemment à terme, les profits.

La gestion de la consommation client

Avec un besoin accru d'exploiter les informations et de les rendre disponibles aux individus, il devient nécessaire de réguler leur utilisation. Dans le cas où l'accès est limité contractuellement, il est utile de suivre la consommation de ses clients, que ce soit en temps d'utilisation, nombre d'appels, volumétrie de ressources, etc. Mettre en place un suivi permettra notamment d'avertir le client lorsque sa consommation atteint un certain seuil limite ou si une anomalie est détectée. Il est alors possible de lui couper l'accès et de lui éviter des surcoûts lorsque la limite est atteinte ou de lui proposer de basculer sur un forfait plus adapté, une pratique courante chez les acteurs des télécoms.



CAS D'UTILISATION « TECHNIQUES »

En complément de la vision métier, un projet Big Data peut également être l'occasion de faire la différence d'un point de vue technique.

Monitoring applicatif

Le monitoring applicatif est souvent considéré comme un projet de prise en main. Accéder aux logs des applications est en effet relativement facile et son contenu est, en théorie, connu et compréhensible par les équipes. Lorsque le volume s'y prête, il est obligatoire de centraliser ces logs pour effectuer un suivi de production et ainsi créer une gestion des incidents, repérer des attaques de sécurité (en analysant les navigations des utilisateurs, par exemple), réaliser un suivi d'anomalies plus efficace, en détectant les erreurs les plus fréquentes et même faire du support prédictif.

La gestion de capacité

La croissance du volume de données exploitées demande davantage de traitements et d'utilisation des ressources et services. En mettant en place des indicateurs de surveillance, d'analyse et de suivi, il est possible d'établir des corrélations entre eux et d'anticiper de nouveaux besoins.
A l'appui de cette analyse, une société d'énergie pourra par exemple prévoir une rapide augmentation ou baisse de la demande, et ainsi réduire le coût de fonctionnement de ses centrales.

Optimisation de la mise à disposition des informations

Bien que les capacités de traitement soient de plus en plus importantes, la possibilité d'exploiter davantage de données pose parfois un problème de temps de calcul. Certaines informations peuvent parfois résulter d'un temps de traitement conséquent. Lorsque ces informations ont une durée de vie très courte et sont donc exploitables pendant un temps limité, le temps de traitement doit être réduit au maximum pour pouvoir y accéder au plus tôt. Dans un contexte où la rapidité de prise de décision est un enjeu de taille (comme pour des produits financiers), cette opportunité est déterminante.

Les différents niveaux du modèle de maturité

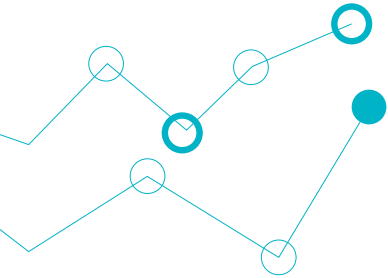
Aujourd'hui, dans les projets Big Data, trois grands niveaux de maturité peuvent se dégager.

Le Néophyte

Le Big Data est à l'état de Proof of Concept (POC), déjà réalisé ou à venir. Dans la plupart des cas, ce niveau de maturité traduit deux visions du Big Data : celui-ci est souvent vu comme une solution rapide pour compléter des manques du SI actuel, ou alors, est envisagé comme une solution technique pour réaliser des projets plus ambitieux, dépassant le cloisonnement du système existant.

Le néophyte ne s'est généralement pas posé les questions suivantes :

- Est-ce que le volume des données est réellement grand ?
- Est-ce qu'il y a assez de données collectées ?



N'ayant pas l'habitude de conserver et traiter toutes les données de son organisation, le néophyte commencera par la mise en place d'outils d'analyse de logs techniques avec les solutions, le plus souvent Open Source, du marché.

L'Initié

Une solution est mise en place, en général sous la forme de Data Lake, agrégeant de nombreuses sources de données ou de petits projets limités à quelques use cases au périmètre réduit. Cette solution est développée en parallèle du SI standard et les deux ont quelques interactions.

Si l'initié possède un niveau plus avancé que le néophyte, il ne s'est encore pas posé les questions suivantes :

- Comment sont intégrées les nouvelles données ?
- Quelle est la granularité de ces données ?
- Comment les filtrer ?

L'Alchimiste

Son projet Big Data est souvent très élaboré et déjà au cœur du système d'information de l'entreprise qui dispose à ce titre d'un Data Lake. Il répond en temps quasi réel aux requêtes métier et provoque la décommission d'applications existantes à son profit.
L'alchimiste aura l'expertise et les compétences nécessaires pour répondre à l'ensemble des questions précédentes. Il saura analyser la granularité des données et les filtrer correctement pour produire des analyses pertinentes, en s'appuyant sur une équipe autonome et experte.

3

Quels sont les coûts d'un projet bien mené ?

Le Big Data a été largement adopté ces dernières années pour des raisons de coûts. En effet, le Big Data est désormais compatible avec le matériel technique d'aujourd'hui et notamment avec des serveurs peu puissants et bon marché.

La réduction du coût de stockage grâce à des disques durs capables de stocker une quantité significative de données est également un facteur clé de cette récente adoption.

Un autre facteur à prendre en compte est que la majorité des technologies Big Data est aujourd'hui basée sur des logiciels Open Source. Certes, le coût d'exploitation de ces solutions n'est pas négligeable, mais reste beaucoup plus accessible dans la phase de démarrage et d'expérimentation. Les retours d'expérience des communautés Open Source facilitent par ailleurs ces étapes et les projets Big Data ont fait émerger de nouvelles communautés avec des personnes extrêmement enthousiastes créant de nouveaux élans.

Reste que dans un projet Big Data, les choix opérés en matière de solutions techniques et de gestion de projet pourront faire varier du tout au tout le coût global de l'opération. Voici quelques éléments à prendre en compte d'un point de vue budgétaire avant de se lancer.

« That which costs little is less valued. »

« On accorde moins de valeur à ce qui coûte peu. »

Miguel de Cervantes, dramaturge

Le coût de la qualité de la donnée

Devant l'immensité et la variété des données (issues de différentes fonctions de l'entreprise : IT, Marketing, R&D, Vente, Support, etc.), le critère numéro un à prendre en compte est la qualité des données.

Dans tout système, la majorité des référentiels comporte des erreurs, souvent humaines, qu'il faut rectifier.
Il faudra alors effectuer des audits de qualité de la donnée qui donneront lieu à des actions de gouvernance pour évaluer la pertinence de ces données et enrichir éventuellement les outils de correction automatique. Dans tous les cas, la qualité des données traitées est à privilégier par rapport à la quantité.

Les coûts d'infrastructure

Pour limiter les coûts d'un projet Big Data, une certaine agilité est attendue de l'infrastructure, en particulier de l'infrastructure de stockage des données. Pour cela, plusieurs solutions existent :

La diversité technologique

Depuis 20 ans, les outils de stockage et d'analyse des données sont en pleine évolution. Cela a commencé avec la problématique de stockage des données et l'apparition de solutions NoSQL en complément (ou parfois en remplacement) des bases de données traditionnelles ; puis ces dernières années avec l'apparition d'outils avancés de traitements en parallèle des données plus simples et accessibles pour les développeurs en lieu et place de solutions ETL. Aujourd'hui, les solutions sont de plus en plus nombreuses avec l'apparition tous les 6 mois de nouveaux produits offrant toujours plus de critères de qualité en termes de capacité de stockage, de sécurité ou de temps d'exécution.

La volumétrie accrue des données

Lorsque nous concevons un système Big Data, il est important d'estimer le volume des données stockées et traitées. Cette estimation est essentielle pour choisir les solutions technologiques adaptées et leurs configurations logicielles sous-jacentes.
Il est également important d'anticiper le volume de données que sa plateforme Big Data pourra supporter dans les mois et années à venir. Il n'est pas rare en effet que tous les 18 mois, le volume des données d'une organisation double.
De nombreuses études estiment que le volume des données des entreprises sera multiplié par 10, d'ici 2020. Ce qui n'est pas très étonnant face à la multiplication des objets connectés.

Les coûts de licence

Devant l'écosystème très éclaté des solutions technologiques, certaines DSI redoutent de ne composer leur plateforme Big Data qu'avec des outils Open Source ayant des niveaux de maturité différents et entraînant des problématiques d'intégration.

En lieu et place, ces DSI préfèrent utiliser des solutions « tout en un » offrant un service répondant à tous leurs besoins et leur assurant un excellent niveau de fiabilité. Ces solutions se composent non seulement de la partie Open Source qui en est le noyau, mais également d'un package Entreprise, constitué d'un ensemble de composants complémentaires pour gérer la surveillance, la reprise de données en cas d'erreur, l'import et l'export massif.

Bien que coûteux en termes de licence, ces produits Entreprise ont atteint un tel niveau de maturité qu'il est devenu dans la majorité des cas, indispensable de les utiliser.

Solution d'Entreprise avec noyau OSS vs Solution 100% propriétaire ?

Les solutions dites d'Entreprise sont composées généralement de différents composants Open source sur lesquels viennent se greffer des composants propriétaires pour faciliter une intégration en Entreprise, en fournissant différents éléments tels que la sécurité, le monitoring et l'administration. On parle alors de solutions « hybrides ». C'est le cas par exemple, de la distribution Hadoop Cloudera. A l'inverse, les solutions 100% propriétaires comportent peu ou pas du tout de composants Open source. Ces solutions sont souvent rigides et peu composables. Une fois mises en place, les organisations sont assujetties pendant des années à la maintenance et à l'achat de licences pour la survie des applicatifs utilisant ces solutions.

Coûts d'apprentissage

Evidemment, les coûts d'apprentissage ne sont pas à sous-estimer. Un projet Big Data, surtout lorsqu'il est mené en interne, nécessite de fortes compétences humaines. Il y a non seulement une montée en compétence à prévoir pour l'équipe de développement mais il faudra également calculer des coûts d'apprentissage pour les autres acteurs, comme les utilisateurs devant s'approprier au quotidien l'outil ou les équipes chargées de mettre en place l'infrastructure. Que la solution retenue exige ou non l'adoption d'un Cloud, la mise en place technique de l'infrastructure par les exploitants prendra du temps. A cette fin, des solutions complètes existent (Cloudera et Hortonworks en proposent), avec un support complet et payant.

Prérequis de fédération et de gouvernance des données

Au-delà des coûts, quelques prérequis à ne pas négliger peuvent eux-mêmes avoir des conséquences sur votre budget.

En effet, comme nous l'avons vu, un projet Big Data est nécessairement transversal et suppose une parfaite coordination entre les différents services, en particulier IT et métier, pour comprendre et valoriser au mieux les données. Il sera, par exemple, indispensable de coordonner les personnes ayant la connaissance des données (qu'on nomme parfois "Data Owner") avec les personnes en charge de valoriser les données comme des "Data Scientists".

Dans le cas où certaines entreprises font appel à de la prestation de services pour les conseiller ou les accompagner, il est essentiel que les consultants des différents prestataires soient immergés afin de travailler main dans la main avec les collaborateurs internes de l'entreprise. Pour aider à atteindre cet objectif, il est préférable d'abandonner les projets en mode forfait, au profit de projets « Agile » avec des cycles itératifs courts permettant d'appliquer rapidement un processus d'amélioration continue. Bien entendu, cette nécessaire fédération des services suppose un accompagnement agile et donc un coût à prévoir dans le cadre de votre budget Big Data.

Au-delà de la fédération, un système Big Data ne peut perdurer sans une gouvernance clairement définie et explicitée des données. L'entreprise devra donc réfléchir aux projets qu'elle souhaite lancer et aux services qu'elle veut optimiser pour définir les données qu'il lui reste à collecter en fonction des besoins opérationnels ou décisionnels identifiés.

Si l'objectif est d'injecter des données de qualité, il faudra dans un premier temps, inventorier les données et leurs sources, puis définir leur qualité (fraîcheur, unicité, conformité, fiabilité…). Sans données qualifiées, les analyses peuvent être erronées et entraîner les entreprises à prendre les mauvaises décisions.

D'autres pratiques de gouvernance peuvent être envisagées comme dans le cas d'un Data Lake centralisant différentes sources de données d'une entreprise. Même s'il existe des signaux faibles dans ce Data Lake, il peut être important de ne pas supprimer ces données pour anticiper de futurs cas d'usage. Le Data Lake devra donc rester cette étendue de données où l'entreprise pourra piocher en fonction de ses besoins.

Le Cloud est-il un prérequis ?

Le Cloud computing est né de la super capacité d'hébergement des grands sites Web. Il apporte une standardisation des infrastructures pour réaliser des calculs multiprocesseurs sur des machines et des systèmes d'exploitation standards, très faciles d'accès.

En tant que telle, ce n'est pas la puissance de calcul des serveurs qui a changé mais la façon de les monter en charge par le client. Auparavant, en cas d'obligation de montée en charge sans budget nécessaire, les DSI avaient tendance à revoir leurs ambitions à la baisse. Désormais, il leur est possible de déployer sur des machines virtuelles, des systèmes développés et conçus pour les traitements de données parallèles. C'est ce que l'on appelle : l'élasticité dans le Cloud, autrement dit, sa capacité à s'adapter aux besoins applicatifs le plus rapidement possible.

« I don't need a hard disk in my computer; if I can get to the server faster... »

« Je n'ai pas besoin d'un disque dur dans mon ordinateur si je peux accéder plus vite au serveur... »

Steve Jobs, en 1997

Des principes adaptés aux besoins Big Data

L'un des principes essentiels du Cloud est le paiement à l'usage. Avec des fournisseurs comme Amazon Web Services (AWS), Microsoft Azure et Google Cloud Platform, il est possible de disposer d'une infrastructure toujours dimensionnée pour répondre au plus juste aux besoins de calcul et au volume de données à traiter.

Les ressources (services ou machines virtuelles) peuvent être allouées et désallouées rapidement, pour s'assurer en quasi temps réel de l'adéquation aux besoins et ne payer que pour ceux-ci.

L'usage du Cloud est une option d'autant plus tentante si vous ne disposez pas de compétences dans l'administration système au sein de vos équipes.

Toutefois, avant d'opter pour un fournisseur de service Cloud, il est nécessaire d'anticiper les contraintes et les risques inhérents à ce choix : le ratio entre les avantages suscités et l'influence qu'aura le Cloud sur le fonctionnement du système d'information doit être clair aux yeux des dirigeants en passe de se lancer dans le Big Data.

Des conséquences à anticiper

De manière assez évidente, la problématique de la sécurité des données ne doit pas être négligée. Les données dans le Cloud doivent être soumises à différentes règles pour assurer à la fois leur confidentialité, leur intégrité mais aussi leur disponibilité à tout moment. Il faudra donc vérifier, entre autres, la conformité des services utilisés avec les différentes législations et normes de la CNIL[1], PrivacyShield, PCI DSS, EU-MC, ISO-27001, lorsque vous choisirez votre hébergeur. Un accompagnement personnalisé et une relation de confiance avec l'hébergeur, couplés à des offres de qualité, doivent permettre d'assurer cette sécurité des données et de lever le principal frein empêchant l'adoption de cette architecture informatique.

Par ailleurs, d'un point de vue technique, basculer sur le Cloud peut s'avérer complexe. Si une brique Cloud doit communiquer avec une partie spécifique du SI, il faudra que cette communication soit sécurisée et puisse accepter des flux, tout en prenant garde au volume de données transitant.

Fort heureusement, les fournisseurs de Cloud ont pris en compte ce genre de contraintes. Ils proposent notamment, en plus des différentes certifications de sécurité, la possibilité d'isoler les ressources à l'intérieur du Cloud (AWS VPC, Azure VNet), de mettre en place des règles de droits d'accès et de gestion très fines (AWS IAM, Azure RBAC), ainsi que la possibilité d'interconnecter de manière sécurisée l'infrastructure on-premise avec les ressources Cloud (Azure ExpressRoute, Amazon DirectConnect).

Une fois ces problématiques prises en compte et leurs risques anticipés, il existe de réels avantages à se diriger vers le Cloud pour concrétiser son projet. Le Cloud offre l'avantage d'une facturation au plus juste et d'une vélocité accrue au profit des équipes.

1 > https://www.cnil.fr/sites/default/files/typo/document/Recommandations_pour_les_entreprises_qui_envisagent_de_souscrire_a_des_services_de_Cloud.pdf

Sur quelle Dream Team compter pour réussir mon projet ?

Une fois que les dirigeants de l'entreprise auront clarifié les objectifs réels d'un projet Big Data et qu'ils auront pu évaluer sereinement les coûts et les gains attendus, il sera nécessaire de constituer une équipe capable de démarrer au plus vite le projet.

Les joueurs clés du match

S'il n'est pas critique d'associer l'ensemble des compétences décrites ci-dessous pour réussir un projet, ces profils ont tous des atouts à faire valoir pour que l'initiative Big Data d'une entreprise devienne plus rapidement une réussite.

DevOps / Sysadmin : au sein d'une infrastructure (Cloud ou pas), la gestion d'une plateforme Big Data (souvent déployée en mode cluster) peut s'avérer complexe et rend généralement la présence de Sysadmins indispensable pour maîtriser correctement le réseau, la sécurité, les différentes configurations des DNS, les règles de routages, de proxy ou autres. Ils seront capables de prendre à leur charge toute la partie installation, administration et réseau. Le rôle d'un DevOps sera de faciliter les livraisons de manière plus fréquentes et de favoriser l'automatisation systématique des différentes chaînes d'intégration.

Data engineers : ce sont des profils développeurs spécialisés dans les systèmes Big Data. Leur rôle est de mettre en place techniquement des outils associés aux plateformes Big Data. Ils connaissent généralement différentes solutions NoSQL comme Redis ou Cassandra, des frameworks de traitements des données comme Spark ou Fkink et la plateforme Hadoop.

Data architect : l'architecte orienté données au sein d'un système Big Data. Son retour d'expérience (y compris si vous restez dans les standards du marché) est très précieux, car la plupart des technologies utilisées sont récentes et ont encore quelques bugs résiduels. Son rôle sera de conserver la cohérence technique et d'expérimenter avant de réaliser des développements à l'échelle. Il doit donc être impliqué au plus tôt dans la phase de conception.

« Se réunir est un début, rester ensemble est un progrès, travailler ensemble est la réussite. »

Henry Ford

Data owner : les Data Owner sont toutes les personnes ayant la connaissance métier des données gérées par une plateforme Big Data. Ces profils ont une connaissance approfondie du métier et/ou de l'organisation et savent mettre en valeur les données et les classifier par niveau d'importance au regard du besoin métier.

Data scientists : leur rôle est de valoriser les données. Souvent mathématiciens de formation, leur savoir-faire leur permet de mettre en œuvre des modèles mathématiques d'analyse. Ce sont souvent des profils rares que les entreprises s'arrachent à prix d'or. Idéalement, les Data scientists ont une forte compréhension du métier mais travaillent main dans la main avec ceux qui comprennent la donnée et savent la structurer : les "Data owner". Tout comme le Data architect, il est possible qu'ils n'interviennent pas à temps plein sur le projet.

Data manager : le but du Data manager est de superviser l'arrivée de données extérieures dans la solution de Big Data. Sur les premières phases du projet, son travail est un temps complet puisqu'il doit avoir une connaissance globale du SI pour isoler les sources de données pertinentes, négocier avec les équipes concernées du temps et des données, superviser le projet d'ingestion et avoir une connaissance fonctionnelle précise des données intégrées. Un Chief Data Officer (le pendant supérieur au Data manager) peut également remplir ce rôle.

RSSI : le responsable de la sécurité des systèmes d'information vous donnera son aval pour la sécurité des données et pour la sélection des briques logicielles à choisir.

Conseiller juridique : ce profil est surtout là pour assurer la qualité de la gouvernance de la donnée dans sa dimension légale. Vous ne pouvez pas stocker de données personnelles sans en avoir informé la CNIL, et avoir eu son approbation.

Les autres atouts de l'équipe

Aussi variée qu'elle puisse être, cette Dream Team du Big Data n'a de sens à être constituée qu'en réponse à une problématique métier bien définie, au risque sinon de gâcher son formidable potentiel. Cette équipe dévoilera donc tous ses atouts lorsqu'elle sera soutenue par un sponsor interne et qu'elle se mettra au service d'un client capable de juger sur pièce de la valeur qui sera créée.

Développer un besoin utilisateur et une analyse des données va poser des questions métier inédites. Il ne sera possible d'y répondre qu'avec des réflexions itératives qui devront s'adapter à de nombreuses réalités marché, technologiques, réglementaires... et qui appelleront à leur tour de nouvelles idées, de nouvelles opportunités et donc, de nouvelles itérations. Au-delà de son expertise et compétence respective, l'atout majeur d'une équipe transversale dédiée à un projet Big Data sera son état d'esprit et son mode de fonctionnement agile. Un parti-pris qui interrogera d'ailleurs rapidement le reste de l'entreprise, si celle-ci n'y a pas été encore confrontée.

6

Que changer dans mon entreprise pour m'assurer de créer de la valeur avec le Big Data ?

Tout ce qui a été dit précédemment nous conduit à une certitude : le Big Data mobilise l'intelligence collective d'une organisation.

En effet, si le Big Data apporte une capacité d'analyse croisée et détaillée des données, permettant de découvrir des informations jusqu'alors inconnues, ces analyses vont nécessiter une collaboration étroite avec les fonctions métier et la mise en oeuvre de KPI adaptés.

« Quand les vents du changement soufflent, certains construisent des abris... d'autres des moulins. »

Proverbe Chinois

Des valeurs et des méthodes

La technique ne peut agir seule, c'est la transversalité qui donnera du sens et de l'objectivité aux données ce qui remet évidemment en cause les modes de fonctionnement classiques des équipes et leur hiérarchie traditionnelle.

Il faudra être agile, en finir avec le fonctionnement en silos qui empêche le partage d'informations et mettre en place des Task Forces transverses permettant de mixer les expériences, les compétences, les cultures pour accroître la diversité et la richesse des analyses.

Par ailleurs, le caractère itératif d'un projet Big Data fait qu'il sera difficile à planifier de A à Z en partant de la supposition que chaque étape se déroulera « comme prévue » et s'enchaînera avec la suivante sans accroc.

En effet, le V de vélocité implique un fonctionnement en mode agile, d'autant qu'au départ, l'entreprise ignore souvent ce qu'elle va faire de la donnée, d'où la nécessité d'itérer le plus souvent possible. Or les pratiques et principes agiles se concentrent sur la validation d'hypothèses le plus tôt possible dans le cycle de livraison, ce qui réduit considérablement les risques d'échec au fur et à mesure que le projet se poursuit.

En travaillant par itération courte, les hypothèses (concernant chaque aspect du projet : code, architecture, données, etc.) peuvent être validées rapidement. Mais lever les doutes et valider les hypothèses ne sont pas les seuls avantages à travailler en mode agile.
L'agilité permet également aux équipes d'un projet Big Data d'avoir un feedback régulier et fréquent, de prendre en compte de nouvelles informations, de nouvelles demandes et dans le cas où certaines hypothèses de départ s'avèrent fausses, de pouvoir réagir rapidement.

L'agilité apparaît donc comme un prérequis indispensable pour ne pas investir dans une mauvaise direction, inciter l'équipe à accepter le changement et l'expérimentation, pour s'inscrire dans une démarche d'amélioration continue.

Il existe un grand nombre de méthodologies basées sur les valeurs du manifeste agile : Scrum, Kanban, eXtreme Programming, etc. Chacune pouvant être appropriée selon le contexte de votre projet Big Data, et il est même possible de créer sa propre méthodologie hybride comme le Scrumban, par exemple. • • •

La méthodologie Scrum, avec son approche itérative, permet entre autres de casser l'effet tunnel. En tant que telle, elle convient parfaitement à un projet Big Data car elle autorise à se tromper, à revenir en arrière et évite ainsi d'avancer dans la mauvaise direction. Les cycles courts s'adaptent aux boucles de feedback des utilisateurs et permettent de les prendre en compte en continu. La méthode offre enfin un cadre permettant de vérifier ou d'invalider des assomptions sans conséquence trop importante sur le projet.

Les fonctionnalités à plus forte valeur ajoutée sont donc développées en priorité et le moindre changement est plus facile à apporter, grâce aux itérations fréquentes et régulières.

La méthode Kanban, quant à elle, se concentre sur la qualité et la fréquence des livraisons. Elle met en rapport la demande et le débit et vise à réduire les sources de variabilité pour augmenter la prédictibilité.

Elle autorise par ailleurs un grand champ d'expérimentations courtes pour améliorer les process, un principe particulièrement adapté aux bases NoSQL sans schéma, sur lesquelles un certain nombre de requêtes préalables sont nécessaires pour identifier celles qui seront utilisées. Du point de vue de la collecte, du traitement, du stockage, de l'analyse et de la supervision, un changement sur un de ces maillons peut impacter les autres. Or la méthode Kanban se concentre précisément sur le suivi du changement de bout en bout de la chaîne de valeur.

Blablacar, célèbre pour son application de covoiturage s'appuyant sur une plateforme Big Data, a choisi de généraliser l'utilisation des méthodes agiles, en adoptant en parallèle Scrum et Kanban. Dans une logique « Production First », l'objectif de Blablacar est de concilier le meilleur des deux mondes, en étant capable de construire des projets entre plusieurs équipes et de les mobiliser pour traiter un problème au plus vite, face aux impondérables qui peuvent survenir en production.

Une nouvelle réflexion sur les KPI

En elles-mêmes, les données sont pratiquement inutiles et se réduisent à de grandes suites de nombres sans contexte. Leur véritable valeur réside dans leur utilisation en conjonction avec des KPI pour produire des analyses qui améliorent les prises de décision et la performance de l'entreprise. Aussi, sans bons KPI, il est impossible d'avoir de bonnes initiatives Big Data.

L'un des premiers indicateurs reste bien sûr la mesure de la satisfaction des utilisateurs, une fois qu'une première version de votre architecture Big Data est en production et qu'elle commence à collecter des retours d'utilisation.

En complément d'une supervision « classique », il est possible d'envisager des indicateurs de performance spécifiques pour une architecture Big Data. Il faudra alors faire preuve d'inventivité pour que ces indicateurs soient suffisamment pertinents et évolutifs, et améliorent la qualité du service rendu.

Il est, par exemple, possible de mesurer une forme de « Time to Achieve Demand » qui correspond à la mesure du temps entre le moment où une expression de besoin est formulée par le métier et le moment où l'architecture a la capacité de produire ce qui est demandé.

On peut également « mesurer » (ou collecter) la fréquence et la qualité des échanges interservices (voire de faire constater la disparition des silos) pour valoriser la décentralisation de l'usage de la donnée et la généralisation des pratiques. Pour mesurer cela, pas de secret : il faudra effectuer des sondages et identifier le nombre de projets inter-silos (ou vraies feature teams) qui existe, et être en mesure de constater clairement la disparition de silos.

Enfin, un autre indicateur possible est la « mesure » (ou le constat) des transformations (et des potentiels de transformation) provoquées par l'introduction du Big Data au cœur du métier. Ces KPI vont alors reposer sur le taux de fidélisation des clients par exemple, les parts de marché gagnées ou tout autre indicateur qui détermine comment évolue l'activité de l'entreprise depuis la mise en place du Big Data.

Chaque projet Big Data doit être lié à des mesures stratégiques clés de l'entreprise. Dans le cas d'une utilisation pour un service Marketing, des changements se verront sur les taux de clics et de conversion, la satisfaction des clients et bien évidemment, sur la marge. Il faudra enfin mesurer comment la communication client et la vision du Marketing ont évolué grâce à l'exploitation de la data.

Il ne s'agit que de pistes pour améliorer la vision Big Data de l'entreprise, dans une optique de réflexion et de co-construction avec les métiers. En effet, il n'existe pas encore à ce jour une « bible des KPI Big Data » dans laquelle chacun pourrait piocher.

Les spécificités des activités et la définition des objectifs particuliers de l'entreprise font que le sujet ne peut être figé : d'où l'importance d'apporter une véritable attention dès le départ à l'émergence de ces nouveaux indicateurs.

Une adaptation aux risques et contraintes du Big Data

Dans le cadre d'un projet Big Data, il sera nécessaire de surmonter rapidement un certain nombre de risques et de difficultés, afin de garantir la rapidité et la qualité des itérations réalisées. Voici les principaux écueils auxquels une entreprise devra faire face et qui nécessiteront parfois une transformation de ses pratiques ou de son organisation :

- **Oublier de prendre en compte les métiers, tous les métiers**

Penser que le métier saura exploiter ses données une fois celles-ci collectées, est une illusion : le métier doit être impliqué dès la phase d'expression des besoins, au même titre que l'IT. Attention toutefois à ne pas réduire un projet Big Data à une boîte à souhaits sur le modèle du « on verra bien ce que l'on en fera », car c'est la plus sûre façon de se suréquiper inutilement. Le mot d'ordre avant de se lancer, est donc de prioriser les besoins métiers.

Redouter l'expérimentation
Par définition, un projet Big Data est la mise en place d'un espace d'expérimentations à destination du métier : c'est un espace dans lequel il est possible d'observer des tendances, de faire émerger des idées mais aussi de tester des hypothèses.
Un projet Big Data est un projet d'expérimentation. Les informations que vous en tirerez pourraient remettre en cause vos certitudes. Il est donc important de ménager du temps pour la production et la veille (bonnes pratiques, retours d'expérience, socle technique).

Négliger les bonnes pratiques
Pour faire un projet évolutif, il faut pouvoir assumer le coût du changement ; il convient donc dès le début du projet, d'anticiper les bonnes pratiques : tests et pratiques de tests, recette (au moins échantillonnée), environnements et pratiques d'intégration continue, de gestion de version, etc.

Se soustraire à la conformité nécessaire
Quoi que vous fassiez avec vos données, vous êtes dans l'obligation de respecter les règles de la CNIL, et bientôt celles du GDPR (General Data Protection Regulation) qui ont été définitivement adoptées par le Parlement européen le 14 avril 2016 et dont les dispositions devront être appliquées dans l'ensemble des 28 Etats membres de L'Union Européenne au printemps 2018.
Ce règlement remplace l'actuelle Directive sur la protection des données personnelles adoptée en 1995.
Pour respecter l'ensemble des réglementations du GDPR, il faudra à terme mettre en œuvre des user stories de votre projet dédiées à la prise en compte des contraintes, sécuriser votre application et y appliquer les mesures obligatoires de communication en cas de faille. Un nouveau métier apparaît depuis quelques temps pour répondre précisément à ces contraintes, celui de Data Protection Officer.
Il est responsable de la protection et de la conformité des données de l'entreprise et doit, pour ce faire, mettre en œuvre les dispositifs informatiques de protection des données et des applications. Son action porte également sur l'organisation de la sécurité, une fonction qui peut être également portée par le Chief Data Officer.

Ignorer le cycle de vie et le volume des données
Big Data ne veut pas dire « tout conserver ». Il s'agit aussi d'anticiper l'obsolescence des données ou les augmentations de volume « prévisibles » et de mettre en place une brique technique qui aura pour but d'historiser ou de supprimer les données obsolètes dans une autre portion de l'architecture.

Oublier de fiabiliser la donnée (et de superviser son architecture)
Sans certitude sur leur fiabilité, 300 pétaoctets de données, ne sont rien d'autre que 300 pétaoctets inutilisables. Ne négligez pas la période de cadrage et les premiers sprints du projet avec des données réelles, pour identifier les incohérences, évaluer l'obsolescence des données importées et mettre en place le cas échéant, les traitements appropriés.

Rester figé dans une organisation inadaptée
Une équipe Big Data va apporter du changement, y compris dans le métier, en matière de gouvernance mais aussi dans la façon dont la donnée pourra influencer l'organisation. Il est donc préférable de vous faire assister par des profils (agiles) dont le métier est d'accompagner le changement.

CONCLUSION

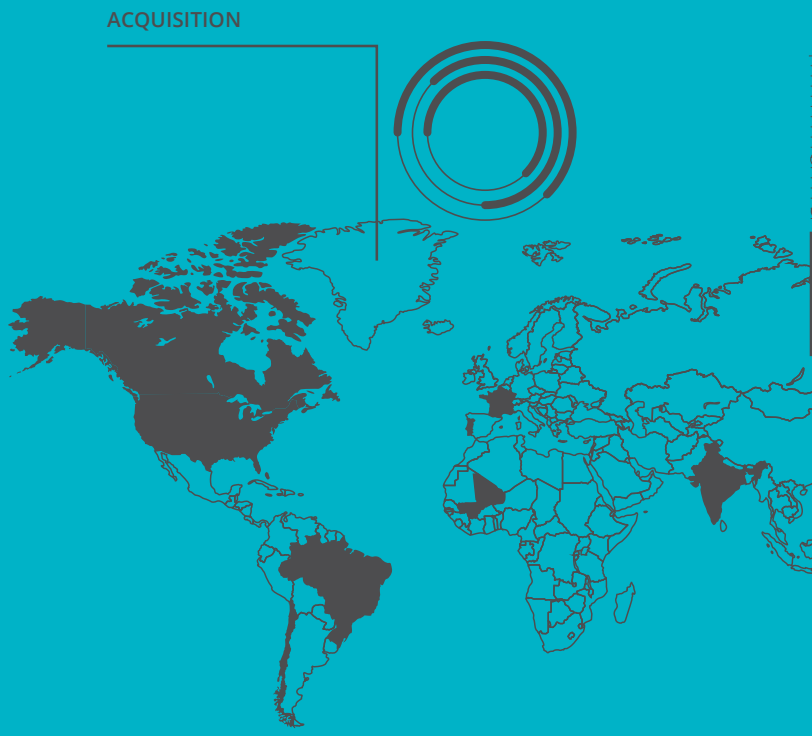
Le Big Data n'est donc pas seulement une problématique de volume à stocker, contrairement à ce que son nom pourrait laisser supposer.

Son principal enjeu est d'abord et avant tout la valorisation de ces données et ce, quel que soit leur volume. Aussi, au-delà de ses composantes technologiques, un projet Big Data a des composantes méthodologiques, juridiques et sociales qui nécessitent un investissement matériel et humain important.

On ne réussit pas à intégrer une démarche Big Data dans la vie et l'activité de l'entreprise sans une conviction forte de sa direction, avec des objectifs business, en termes d'expérience client ou de productivité, explicites. La mise en musique de cette vision pourra alors être réalisée par une équipe transverse, réunissant de nombreuses compétences et à même de prendre en compte la diversité des enjeux économiques et organisationnels de tels projets.

C'est avec cette feuille de route claire qu'il sera ensuite possible de maîtriser au mieux les enjeux techniques du Big Data. Une ambition à laquelle nous consacrons la seconde partie de ce livre blanc.

Quel écosystème technologique ?



Le volume des données stockées ne cesse de croître, tout comme celui des connexions entre ces données. Une telle expansion du volume de connexions est en partie liée aux architectures de plus en plus orientées vers les APIs et à la multiplication des objets connectés.

Pour appréhender et gérer cette nouvelle complexité, les développeurs et architectes des équipes opérationnelles s'appuient le plus souvent sur des solutions clés en main.

Cette seconde partie de notre livre blanc présente les différentes technologies d'une plateforme Big Data et s'appuie sur notre expérience terrain, pour vous guider dans leur choix et leur utilisation.

Elle s'adresse à tous ceux qui souhaitent mettre en production un système de données Big Data ou se tenir informé des architectures et des technologies les plus utilisées. Elle s'attache également à répondre aux problématiques les plus fréquemment rencontrées dans le cadre d'un projet Big Data.

Lorsque la donnée est injectée dans un système Big Data, elle suit un cycle de vie bien particulier dont le cheminement pourrait se résumer ainsi : l'acquisition de la donnée, son éventuel traitement au moment de son injection, le stockage de cette donnée ou d'une donnée dérivée, son traitement, puis sa restitution et son analyse pour aider les métiers à la prise de décision. C'est précisément ce cheminement que suivra la seconde partie de notre livre blanc.

P.38 > 1 Acquisition / Extraction

P.41 > 2 Stockage

P.50 > 3 Flux d'événements

P.54 > 4 Intégration des données et traitement

P.62 > 5 Analyse et Restitution (Visualisation)

P.64 > 6 Architectures des systèmes Big Data

P.71 > 7 Infrastructure Big Data

P.75 > Conclusion Partie II

Acquisition / Extraction

Les données entrent dans un système Big Data à travers différentes « portes » qui peuvent être un ensemble de fichiers déposés sur un système de fichiers (pattern nommé “File Transfer”), via **SQOOP** ou encore via des systèmes de collecte comme **Flume**, dans l’écosystème **Hadoop**, ou même **Logstash** dans l’écosystème **ElasticSearch.**

Au sein d'un système Big Data, les sources de données sont souvent multiples :

- Vidéos
- Transactions d'une plateforme e-commerce
- Audios
- Images
- Données de collecteurs IoT
- Médias sociaux
- Logs applicatifs
- Informations des réseaux sociaux.

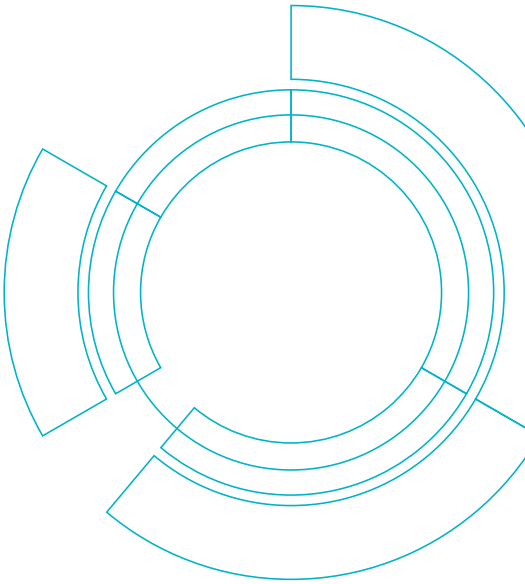
L'intégration de données via un système de fichiers

Apparus dans les années 70, la majorité des composants logiciels s'échangent des informations à travers des fichiers de données. Ce type d'intégration est très pratique et très simple à mettre en place via la mise à disposition d'APIs dans les différents langages de programmation. Cette intégration de données sous forme de fichiers souffre cependant de nombreux problèmes de sécurité, de temps d'intégration, de reprise sur erreur et de cohérence transactionnelle.

Acquisition via un Data Lake

Un Data Lake a pour vocation de regrouper toutes les données de plusieurs sources hétérogènes à un même endroit pour être accessibles au plus grand nombre.

Pour garantir le succès de la mise en place d'un Data Lake, il est souvent nécessaire de mettre en place des phases de prétraitement des données dédiées à la compréhension, à l'exploitation et au nettoyage de ces mêmes données. Ces phases, généralement chronophages et fastidieuses, sont pourtant indispensables à l'obtention de résultats exploitables et pertinents pour le domaine métier.



Extraction des données via les ETL

Les ETL (Extract, Transform, Load) permettent d'alimenter un datawarehouse à partir de différentes sources de données, ou de synchroniser des systèmes à travers différents batchs. Ces ETL offrent des fonctionnalités d'extraction, de transformation (rapprochement, format, dénomination, calculs) et de chargement des données.

Les données du datawarehouse sont généralement des données hétérogènes et plus ou moins complexes. Les ETL manipulent ces données et les ré-injectent dans une base de données relationnelle afin de les homogénéiser, ce qui permet de simplifier leur utilisation par les composants de traitement. En raison de ces caractéristiques, les ETL sont souvent utilisés dans un système Big Data.

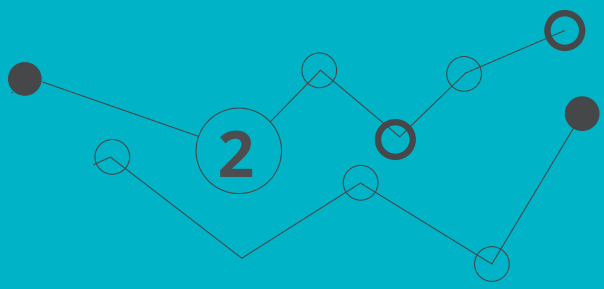
En France, l'ETL Open source Talend prend une part de marché importante et propose, entre autres, des connecteurs de type : SGBD, fichiers, solutions NoSQL, Hadoop ou Spark.

L'émergence de la solution Apache Kafka

Il n'est pas rare désormais de voir l'outil **Apache Kafka** comme une solution d'injection des données au sein d'un système Big Data.

Kafka se définit comme un système de message distribué et résilient, supportant en entrée un débit très important d'écritures de données. Kafka a été créé pour des besoins de forte volumétrie de la plateforme Linkedin.

Ses caractéristiques techniques en font un outil souvent adapté aux besoins d'acquisition de données massives dans le cadre de projets Big Data.



Stockage

Le leadership des SGBDR

Les éditeurs de **SGBDR** (Système de Gestion de Bases de Données Relationnelles) sont des acteurs bien installés depuis plus de 20 ans. Leur situation de leardership tient principalement au langage standardisé **SQL** (Structured Query Language) offrant un apprentissage plus rapide et plus simple pour les développeurs. Cette facilité d'apprentissage explique sans doute l'échec des bases de données orientées objets qui ont fleuri autour des années 2000 et n'ont jamais véritablement percé, malgré une approche novatrice.

Les bases de données relationnelles

Les systèmes de gestion de bases de données relationnelles (SGBDR) dominent largement le marché, au détriment du SGBD objet. Cette domination s'explique par l'aboutissement d'évolutions successives avec l'apparition de systèmes à base de fichiers, puis des bases de données hiérarchiques et des bases de données réseau. Depuis plusieurs dizaines d'années, ces bases de données relationnelles ont gardé une grande stabilité malgré l'évolution des langages de programmation, des changements d'Architecture, de plateformes ou de processus.

A ce jour, l'utilisation des SGBDR reste privilégiée par les équipes de développement. Pour s'interfacer avec ces systèmes de bases de données, un vaste écosystème de frameworks techniques et d'outils cohabitent tels que des IDEs, des briques d'APIs ou des outils d'exploitation et de sauvegarde, érigeant les SGBDR en véritable standard.

Schéma de base de données

L'une des principales caractéristiques de ces bases de données relationnelles est l'application d'un schéma de données, autrement dit d'une spécification décrivant la structure des données qui seront stockées. Les données sont alors organisées en tables afin de réduire leurs éventuelles anomalies (telles que l'inconsistance des données ou le non respect d'invariants).

L'ensemble de ces pratiques est appelé la normalisation des données. Les données sont ainsi conçues indépendamment de la façon dont elles sont requêtées. Pour satisfaire la variété de ces données, une opération de jointure est régulièrement mise en place. Cette opération est fréquemment exploitée par les utilisateurs, en raison de la très grande facilité d'utilisation du langage de requête standard SQL (Structured Query Language) avec son opérateur 'join'. Cependant, dès que la taille des tables de données augmente, l'opération de jointure devient trop coûteuse. Par ailleurs, cette opération a été conçue en mode non distribué provoquant des problématiques lorsque les données sont distribuées sur plusieurs serveurs.

Gestion des transactions

Des propriétés de transactions ACID (Atomicité, Cohérence, Isolation et Durabilité) sont associées aux bases de données relationnelles.

Dans un système distribué, les transactions sont des transactions distribuées respectant la spécification XA permettant d'assurer l'atomicité de la transaction entre plusieurs serveurs. On parle alors de transactions Two-Phase-Commit (2PC). Les Transactions XA sont plus longues puisque la mise à jour de la donnée est retardée jusqu'à ce que chacun des intervenants entre en jeu.

Les systèmes distribués sont des systèmes complexes pouvant mener par exemple, en langage Java, à des exceptions pour les applications comme des exceptions "HeuristicException", difficiles à gérer d'un point de vue programmation.

Inconvénients des bases de données relationnelles

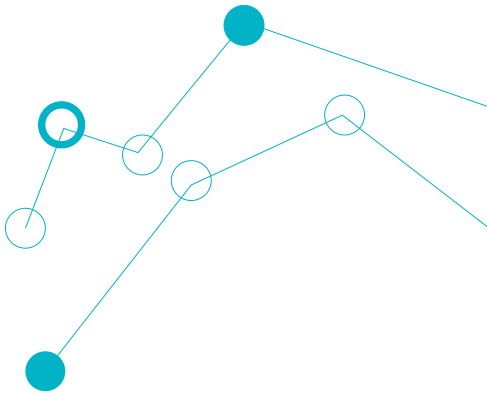
Reste que la majorité des bases de données relationnelles ont leur limite et peuvent présenter quelques problèmes, dont les principaux sont :

- L'usage d'un schéma souvent trop rigide ne répondant pas à tous les besoins. En effet, un schéma de données rigide n'est pas adapté dans le cas où les données injectées sont très hétérogènes en raison d'un manque de maîtrise ou de connaissance de la source.
- Une scalabilité horizontale souvent trop difficile à atteindre. Les bases de données relationnelles sont en effet conçues majoritairement pour s'exécuter sur une seule machine, nécessitant parfois l'acquisition d'une machine plus performante (RAM, Disk, I/O, etc.). Au-delà de la difficulté à maintenir une continuité de service, il est aujourd'hui moins coûteux d'avoir plusieurs machines en réseau. Si les machines sont moins performantes individuellement, leur mise en réseau offre une performance globale supérieure.
- Des problèmes de correspondance entre un modèle métier (en mémoire) et un modèle de données, regroupés sous le nom « d'impedance mismatch ». Pour éviter des frustrations, la majorité des développeurs logiciels ont recours à des frameworks de correspondance objet-métier et objet de données, tels que les outils ORM (Object Relational Mapping) comme Hibernate ou EclipseLink ou avec d'autres approches comme jOOQ.
- Des problèmes potentiels de coûts d'achat massif de licences.

Les premiers acteurs à se confronter aux limites des SGBDR furent les Géants du Web tels que Google, Facebook, Twitter ou LinkedIn. Pour ces acteurs, les besoins majeurs sont :

- la capacité à distribuer des traitements sur un nombre important de machines,
- la capacité à répartir des données entre un nombre important de machines,
- la gestion d'une très forte volumétrie,
- la capacité à gérer des données pouvant être très hétérogènes,
- la distribution des données sur plusieurs datacenters.

C'est précisément pour répondre aux exigences de ces acteurs du Web, que les bases de données NoSQL ont été créées et cohabitent désormais avec les bases de données traditionnelles.



Écosystème des bases de données NoSQL

C'est suite à la publication du célèbre livre blanc de Dynamo en 2007 (Amazon's Highly Available Key-value Store, SOSP 2007) que l'on s'est intéressé de près aux bases de données NoSQL basées sur un système distribué. Il existe désormais une grande variété de bases de données NoSQL avec pour la plupart, très peu de différences. Majoritairement Open Source, ces nouvelles solutions sont conçues pour pouvoir s'installer, se configurer et s'exécuter sur un cluster de serveurs.

Se distinguant complètement des bases de données relationnelles, ces bases de données, peuvent être catégorisées de la manière suivante :

- Les bases de données NoSQL de type Clé/Valeur

  Avec une base de données Clé/Valeur, chaque morceau de données est stocké dans une collection unique et compartimentée avec une clé unique, ayant un rôle de point d'accès principal, pour accéder à l'objet de la donnée. A une clé, de type chaîne de caractères, on associe une valeur. Cette valeur peut être une chaîne de caractères ou des structures de données plus complexes comme avec les solutions Redis ou Cassandra. En termes de cas d'utilisation, on retrouve des solutions de cache, de stockage de données temporaires comme des sessions utilisateurs Web, de données d'analyse ou une gestion "ad-hoc" pour représenter des données temporelles.
- Les bases de données NoSQL orientées Colonne

  Les bases de données orientées Colonne sont constituées du concept de Colonne et de Famille de colonnes. Une colonne est une entité représentant un champ de données. Chaque colonne est définie par un couple clé/valeur. A noter qu'une colonne contenant d'autres colonnes est nommée super-colonne. La famille de colonnes est un conteneur permettant de regrouper plusieurs colonnes ou super-colonnes. Les colonnes sont regroupées par ligne et chaque ligne dispose d'un identifiant unique. En termes de cas d'utilisation, on retrouve l'ensemble des cas d'utilisation possibles avec une base de données clé-valeur, mais également des données plus complexes de structuration.
- Les bases de données NoSQL orientées Document

  Une base de données orientée Document est très similaire à une base de données clé-valeur, dans la mesure où elle est constituée d'une collection de documents.

  Un document est composé de champs et de valeurs associées qui peuvent être requêtés.
  Parce qu'elles bénéficient d'une conception plus simple que les bases de données relationnelles, ces bases conservent des performances généralement élevées.

  En raison de leur modèle, ces bases sont très utilisées dans le milieu du développement de CMS (Content Management System) et de gestion de contenu. Dans cette catégorie, la solution la plus populaire est MongoDB, et vient ensuite mais moins célèbre, OrientDB. Avec ses structures de données avancées, Redis peut également être assimilé à une base de données Document.
- Les bases de données NoSQL orientées Graphe

  Inspirées de la théorie des graphes, les bases de données Graphe s'appuient sur un moteur de stockage pour les objets (généralement sous forme de Document) et sur des mécanismes d'arcs (munis de propriétés) pour décrire les relations entre les objets.

LA CATÉGORISATION DU NOSQL

La classification en catégorie des solutions NoSQL est de moins en moins respectée face l'évolution de chaque solution avec des produits de plus en plus autonomes en termes d'Architecture, de configurations ou même en termes de langage de requête.

Mais au-delà de cette catégorisation, dans le cas des systèmes distribués, le théorème **CAP (Consistency, Partition, Tolerance)** a permis de rajouter un critère de catégorisation avec un compromis entre la disponibilité et la consistance des données lorsqu'un problème survient comme l'indisponibilité d'un serveur d'un système distribué.

LE NOSQL FACE AUX LIMITES DES SGBDR

La majorité des solutions NoSQL permettent de faire face aux limites des systèmes de **SGBDR** de certains cas d'usage, comme la gestion d'objets hétérogènes. Par exemple, avec des solutions comme **MongoDB**, il n'est plus nécessaire de déclarer au préalable des champs représentant l'objet à stocker. Cela amène une plus grande flexibilité dans les modèles de données, ainsi qu'une simplification de la modélisation. Ces solutions sont dites "**schemaless**" signifiant que les contrôles du côté de la solution de stockage en termes de gestion des données, sont réduits ou absents. Certains développeurs pensent que le terme "**schemaless**" signifie que les contrôles de validation des données ne sont pas nécessaires, ce qui est une interprétation erronée dans la mesure où les données doivent toujours être validées du côté applicatif à travers un mécanisme programmatique.

Gestion de la temporalité

Avec l'arrivée de nouveaux cas d'usage comme l'Internet des objets (**IoT**), il est utile de pouvoir mettre le concept de temporalité au coeur de la modélisation de la donnée. Certaines solutions comme **MongoDB, Redis et Cassandra** proposent des structures de données permettant de modéliser des données temporelles. Cependant, la gestion du temps n'est pas native au coeur de ces produits. Ainsi, des solutions comme **InfluxDB**, **Druid** ou **Riak TS** sont apparues proposant de véritables fonctionnalités basées sur le temps. Ces solutions sont généralement dotées d'un langage de requête dédié sur la temporalité pour faire par exemple, des comparaisons entre deux dates.

Pourquoi choisir une base de données NoSQL ?

Au delà des nouveaux cas d'usage présentés ci-dessus, les bases de données **NoSQL** peuvent offrir une alternative aux **SGBDR** permettant une meilleure scalabilité horizontale, un débit de lecture plus important et la tolérance aux pannes.

Chaque solution **NoSQL** possède des caractéristiques propres et ses avantages.

Côté architecture des solutions **NoSQL**, les architectures Master-Slave et peer-to-peer se distinguent :

- Architecture Master-Slave

Les architectures Master-Slave sont composées de noeuds dit Maître (Master) qui ont la responsabilité de servir l'ensemble des lectures et des écritures en provenance des clients de la solution NoSQL ; et d'un ensemble de noeuds Esclave (Slave) qui ont uniquement la responsabilité de lecture. • • •

Cette topologie de déploiement est présente dans des produits très populaires comme **Redis** ou **MongoDB**.

Il existe plusieurs terminologies : Le noeud maître (master) est aussi appelé « leader » ou noeud primaire (primary), les autres noeuds sont des répliques (replicas) et peuvent être également nommés "followers". On parle alors de "Master-Slave" ou "leader-follower".

Dans cette topologie d'Architecture, chaque écriture s'adresse au leader. En revanche, les clients en lecture seule peuvent utiliser soit le leader, soit le follower. Reste que le follower est asynchrone, ce qui peut fournir des données qui ne sont pas complètement mises à jour. Ceci est extrêmement important dans les scénarios métier.

■ Architecture Peer-to-peer

Les architectures peer-to-peer ont été conçues pour être complètement distribuées et ne pas présenter de point de défaillance unique (en anglais Single Point of Failure ou **SPOF**). Dans cette topologie d'Architecture, tous les noeuds sont équivalents en ayant le même niveau de responsabilité. Les noeuds sont organisés sous la forme d'un anneau, et en cas de défaillance, le noeud est simplement retiré de l'anneau. Lors de l'ajout d'un nouveau noeud, le moteur **NoSQL** doit connaître des noeuds stables (en anglais « seed »). Cette topologie est présente dans les produits très populaires comme **Cassandra** ou **Riak**.

LE RETOUR D'UN LANGAGE DE TYPE SQL DANS LE NOSQL

Pendant de nombreuses années, les bases de données NoSQL se sont distinguées par leur capacité à être flexible et à proposer des solutions de scalabilité linéaire, parfois au détriment d'un langage de requête sophistiqué et accessible à tous les intervenants d'un projet Big Data (majoritairement développeurs et exploitants).

C'est le cas, par exemple, de la base de données **MongoDB** qui se distingue par un langage de requête évolué mais en JavaScript, langage présent uniquement dans la communauté des développeurs. Ainsi, **MongoDB** a souvent nécessité une courbe d'apprentissage importante de la part des exploitants, peu familiers au JavaScript, afin de s'approprier pleinement les interactions avec MongoDB.

Depuis quelques années, le **SQL** revient en force. Ses utilisateurs ont ainsi renoué avec ses avantages, sa concision et les performances des techniques d'optimisation de ses requêtes matures.

Pour suivre cette mouvance, par exemple, Cassandra s'est doté d'un langage de manipulation et de requête des données, le **CQL** (pour Cassandra Query Language). Très proche du SQL et des bases de données relationnelles, cela a permis aux développeurs et aux exploitants d'être dans un environnement plus familier.

SCHÉMA DES DONNÉES NOSQL

En termes de conception des données, les bases de données relationnelles nécessitent la mise en place au préalable, d'un schéma de données. Concernant les bases de données NoSQL, le terme "schemaless" est souvent utilisé. Cela ne signifie pas pour autant qu'il n'y ait pas de schéma de données, et dans le cas d'une migration par exemple, un schéma restera utile, voire indispensable. De même un système de stockage plus souple ne signifie pas nécessairement une phase de modélisation raccourcie ou simplifiée.

LES LIMITES DES SOLUTIONS NOSQL

En résumé, les avantages des bases de données NoSQL sont : la possibilité d'avoir une scalabilité massive, une haute disponibilité, des coûts de licence faibles et une forte flexibilité du schéma des données. En revanche, les bases de données NoSQL offrent une capacité de requêtage limitée et peu de possibilités de standardiser.

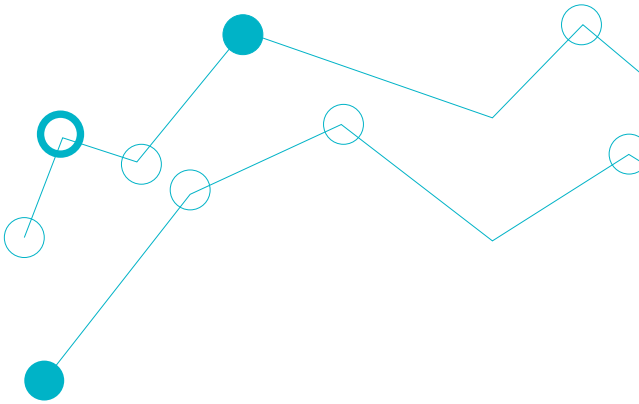
Ainsi, malgré l'explosion des solutions NoSQL, elles ne sont pas nécessairement adaptées à tous les usages. Dix ans après la genèse du mouvement NoSQL, la majorité de ces solutions continuent à évoluer en termes d'architecture et d'outillage.

DÉNORMALISATION NOSQL

Contrairement aux SGBDR, au sein d'un système Big Data, la mouvance NoSQL prône une approche guidée par l'utilisation qui est faite des données. Le paradigme "Query Driven Design" est souvent utilisé.

En termes de modélisation, dans certains cas, les données seront embarquées dans une donnée parente. C'est notamment le cas lorsque le volume de la donnée est faible et que celle-ci est peu mise à jour, que cela nécessite une lecture rapide. Cette modélisation est généralement regroupée sous la forme d'unités indépendantes ou d'« agrégats ». La jointure entre ces données se fait alors côté applicatif via des identifiants.

Cette dé-normalisation est dans la plupart des cas favorisée pour atteindre des gains de performance en s'adaptant, au plus près, aux solutions utilisées.



Système de fichiers distribués

L'usage des systèmes de fichiers distribués répond à un besoin lié à des bases de données volumineuses. Moins coûteux que des architectures de type SAN, il existe des systèmes de fichiers distribués dont les plus connus sont Lustre et MogileFS.

Ces systèmes de fichiers ont des performances hautes et sont généralement utilisés dans des supercalculateurs, que l'on retrouve généralement dans des systèmes météorologiques, des plateformes de simulation pétrolière ou dans le domaine de la finance.

Processus applicatif distribué

Dans le cas de volume de données important, il peut apparaître que les traitements ne soient plus réutilisables par un seul et même noeud applicatif. Pour pallier cette contrainte, il faut répartir des processus sur un ensemble de machines. Il existe des outils comme les environnements Hadoop dans l'écosystème Java, Akka, NodeJS dans le monde JavaScript. Il est également possible d'utiliser des langages comme Erlang.

HADOOP

Hadoop est un vaste écosystème permettant le traitement massif de données en mode Cluster, allant d'une à plusieurs centaines de machines. Écrit en Java, il a été créé par Doug Cutting et Michael Cafarella en 2005 (après avoir créé le moteur de recherche Lucene, Doug travaillait alors pour Yahoo sur son projet de crawler web Nutch). Hadoop est un framework libre Open Source qui va gérer la distribution des données au cœur des machines du cluster, leurs éventuelles défaillances, mais aussi l'agrégation du traitement final. L'architecture est de type « Share nothing » : aucune donnée n'est traitée par deux nœuds différents, même si les données sont réparties sur plusieurs noeuds (principe d'un noeud primaire et de noeuds secondaires).

Hadoop est composé de quatre éléments :

- Hadoop Common : ensemble d'utilitaires utilisés par les autres briques Hadoop.
- Hadoop Distributed File System (HDFS) : un système de fichiers distribués pour le stockage persistant des données.
- Hadoop YARN : un framework de gestion des ressources et de planification des traitements.
- Hadoop MapReduce v2 : Un framework de traitements distribués basé sur YARN.

Les points forts d'Hadoop sont sa capacité à traiter des volumes de données très importants, sa sécurité, son mode de distribution et sa tolérance aux pannes. En environnement Hadoop, malgré des traitements parallèles, le volume de données est tellement important que le temps d'exécution demeure conséquent.

La majorité des équipes de développement mettent en place Hadoop pour introduire des logiques de traitement, d'indexation et des problématiques de recommandation.

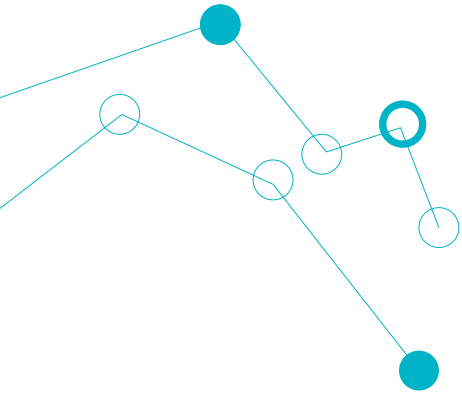
HADOOP DISTRIBUTED FILE SYSTEM (HDFS)

HDFS est devenu un standard de facto pour le stockage de données volumineuses sur un cluster de machines.

MAPREDUCE

MapReduce est une technique de programmation distribuée très utilisée dans les solutions NoSQL qui consiste à distribuer les traitements (les requêtes) sur plusieurs machines. Cette technique est généralement composée de deux étapes :

- **Étape de Mapping** : Par exemple, à chaque mot trouvé, on associe une valeur 1 ou une liste de UserId/User, et on renvoie à nouveau un couple UserId/User dont les propriétés de l'objet User correspondent à certains critères. Cette étape est généralement exécutée en parallèle entre les différentes machines.
- **Étape de Réduction** : A partir du résultat de l'étape précédente, on applique une opération de réduction, en faisant, par exemple, la somme des éléments pour compter le nombre d'éléments répondant à un critère donné.



Recherche distribuée

La recherche est souvent un élément clé sur un système distribué. On utilisera le plus souvent des moteurs de recherche dédiés comme **ElasticSearch** ou **SolR**. Le principe de base est de construire un index de recherche distribuée. Dans tout système, chaque index occupe un certain espace (disque ou mémoire) et est donc coûteux. Il est généralement indispensable de trouver le bon compromis entre ce qui est nécessaire d'indexer et l'espace induit à gérer. Connaître au plus tôt comment les données seront accessibles ("query-driven-design") est donc souvent utile.

Configuration distribuée

Dans le cas d'un système distribué, il faut également adresser la distribution de la configuration (élément à ne pas négliger). Celle-ci est souvent incluse nativement dans certains outils ou se base sur des bibliothèques dédiées comme **Apache Zookeper** dans le cas, par exemple, de **Apache Kafka**.

CAS D'UN DATA LAKE POUR LA SOLUTION DE STOCKAGE

Côté persistence, un Data Lake qui contient généralement des milliers de fichiers, repose classiquement sur HDFS. En termes de format de stockage, des formats binaires récents tels que Avro, Parquet ou ORC pourront être utilisés pour bénéficier de meilleures performances dans certains traitements de données.

3

Flux continu d'événements

Le Big Data pourrait se résumer en deux grands axes : le premier serait la possibilité de stocker massivement les données, autrement dit, les Data Lakes, et le second serait la possibilité de traiter les données dès qu'elles entrent dans le système Big Data, ce qui correspond aux event streams, ou en français, aux flux continus d'événements.

L'idée de structurer les données sous forme d'un flux, permet de tracer tout ce qui se déroule (les faits passés) ou de décrire tous les changements sur une base de données.

Malgré des principes souvent similaires, le flux continu d'événements peut avoir des significations différentes en fonction des usages, portant à confusion.

Parmi les différents termes techniques qui utilisent l'idée des **Event Stream Processing**, nous trouvons l'**Event Sourcing, Reactive, CEP, Stream Processing, Actors/SEDA, Change Capture**, particulièrement utilisés chez les géants du Web. A titre d'exemple, le produit Google Analytics fournit à ses utilisateurs un composant JavaScript à intégrer sur leur site Web, pour capturer les pages vues par les visiteurs, permettant à l'administrateur du site d'explorer les données des visiteurs par période de temps, par URL de pages, etc.

Fonctionnement d'un flux continu d'événements

Un flux continu d'événements consiste à tracer un événement à chaque fois qu'un utilisateur du site regarde une page. Parmi les informations tracées, on peut avoir un timestamp, une adresse IP du client, la sessionID et l'URL de la page consultée. Cet événement est « **immutable** » c'est-à-dire qu'il ne pourra être modifié et contient factuellement tout ce qui s'est passé.

Se pose alors la question de ce que nous faisons de ces événements. Deux options sont possibles :

- **Stocker ces événements tels quels dans une base de données** (avec le bon dimensionnement) ou un data warehouse ou un cluster Hadoop. L'analyse des données consiste alors à exécuter une requête de type "big SELECT" sur les données. Grâce au langage SQL, il est possible de grouper les données par URL, par période de temps ou de filtrer sur certaines conditions pour obtenir par exemple, le nombre de pages vues avec un COUNT(*). Ce type de requête scanne tous les événements ou une partie des événements, puis lance une agrégation à la volée.

Exemple de requête : SELECT url, COUNT(*) from events WHERE ... GROUP BY url

- **Stocker des données des événements agrégés** comme stocker directement le nombre de pages vues à travers un objet compteur. Ce compteur sera incrémenté pour chaque page visitée. Cependant et afin de répondre au mieux aux requêtes des utilisateurs, il est nécessaire de stocker plusieurs compteurs, souvent dans un cube **OLAP** (un cube à plusieurs dimensions où dans notre exemple une dimension pourrait être l'URL, une autre serait le temps de l'événement et une autre pourrait être la version du navigateur Web, etc.). Ainsi, avec un cube **OLAP**, si nous souhaitons connaître le nombre de pages visitées pour une URL donnée, il suffit de lire le compteur qui est à la combinaison de l'URL et d'une date donnée. Il n'est donc pas nécessaire de parcourir tous les événements pour obtenir le résultat, car la lecture d'une seule valeur suffit.

Événements bruts versus Événements agrégés : des cas d'utilisation différents

Le stockage brut des événements offre mécaniquement plus de flexibilité en termes d'analyse car il n'est pas nécessaire de connaître à l'avance les requêtes qui vont être manipulées. Par exemple, en **mode offline**, à partir des événements stockés, il est possible d'en déduire le parcours utilisateur lors d'un achat en extrayant la liste des pages. Cette information est très utile pour les sites d'e-commerce, d'un point de vue Machine Learning, afin d'optimiser le processus d'achat en sachant, par exemple, que les personnes qui achètent le produit X, achètent aussi le produit Y.

• • •

En revanche, si votre besoin est de pouvoir réagir en temps réel pour éviter une attaque par ensemble de requêtes massives de votre site web, la solution du stockage de données agrégées est plus appropriée. En effet, implémenter ce dernier cas d'usage avec le stockage d'événements bruts obligerait à re-parcourir en permanence (pour chaque requête) la liste des événements dans le but de savoir si une personne a dépassé le seuil limite.

Il est donc plus efficace de garder un compteur du nombre d'utilisateurs par adresse IP et par heure.

Pour des systèmes d'alertes ou de finance de marché où le besoin de réactivité est particulièrement fort en termes d'analyse de données, le stockage de données agrégées est ici à privilégier.

A noter cependant que les solutions de stockage modernes (par exemple NoSQL ou base de données relationnelles) offrent de réelles capacités de parcours sur des volumes de données importants. Ainsi, il est important d'être pragmatique dans le choix d'Architecture et de mettre en place des campagnes de performance au plus tôt des projets.

UN ÉVÉNEMENT EST UN ÉVÉNEMENT MÉTIER DANS LE PASSÉ

Un événement métier représente un événement passé qui s'est produit dans le système d'information et qui a un impact sur votre métier. Il s'agit d'objets qui ne changent pas et qui favorisent le découplage des systèmes à travers des flux d'événements (stream events). Pour faire le parallèle avec le Domain Driven Design (DDD), les événements métiers sont appelés des "Domain Event" modélisant un concept métier chronologique du Domain Model. Côté implémentation, les événements sont des structures de données qui ne changent pas (dit immutables) comprenant comme attribut au moins un identifiant, la date de l'événement et ensuite des informations représentant le contexte.

REDIS : UNE SOLUTION TRÈS ADAPTÉE POUR LE STOCKAGE D'AGRÉGATS D'ÉVÉNEMENTS

Au-delà de sa très grande simplicité d'usage et de son excellente stabilité, Redis est très adapté au stockage de données représentant des événements métier, fournissant un objet technique de stockage « atomic counter » et la possibilité de modéliser simplement des périodes de temps au niveau des clés. Par exemple, à chaque fois que le serveur Web traite une requête, ce serveur envoie une commande d'incrément (+1) à la solution de stockage Redis avec une clé matérialisant l'IP et l'heure courante.

Exemple Redis: redis-cli > incr 123.5.22.4:13:00

Redis propose également une implémentation de l'algorithme HyperLogLog permettant d'obtenir une approximation du nombre d'éléments uniques, le tout, en utilisant un espace mémoire petit et constant. Redis permet cela en fournissant une structure de données occupant 12 Kbytes avec un pourcentage d'erreur de seulement 0.81%.

PLUSIEURS NOMS POUR LE STOCKAGE DES STREAMS D'ÉVÉNEMENTS

En fonction des besoins et des solutions plus moins sophistiqués, plusieurs termes sont proposés, tels que **event stream, message queue ou event log**.

Du côté de la communauté **DDD** (Domain Driven Design), le terme **Event Sourcing** est particulièrement plébiscité car connu par les développeurs d'applications. Event Sourcing se concentre sur la façon de structurer les données dans les bases de données. Le principe est de ne pas stocker l'état d'un objet qui peut être mis à jour, mais de stocker individuellement tous les changements qui se déroulent sur une base de données (relationnelle, NoSQL ou autre). Garder tous ces changements d'état comme un log d'événements immutable évite de perdre l'historique de la valeur d'un objet et fournit beaucoup d'informations utiles, notamment pour du traitement d'analyse.

CAS DE LA PERFORMANCE LECTURE / ÉCRITURE STREAM PROCESSING

Il est généralement observé que les besoins d'optimisation pour la lecture sont très différents de l'optimisation pour l'écriture.

C'est la raison pour laquelle, il est recommandé de séparer la modélisation des données en lecture, de la modélisation des données, en écriture. Ce paradigme est l'idée même du pattern d'Architecture **CQRS** (Command Query Responsibility Segregation).

4

Intégration des données et traitement

Dans un système Big Data, ce sont les connecteurs logiciels qui sont en charge d'acquérir et de transformer les données, afin de les stocker dans une forme répondant aux exigences métier du système, pour être utilisées.

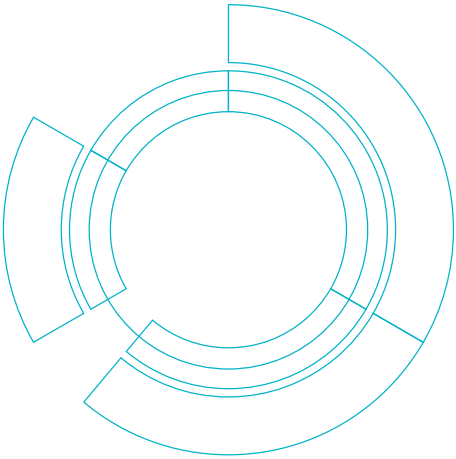
Du point de vue de l'architecture de la donnée, plusieurs solutions de stockage sont mises en place afin d'avoir différentes formes de données correspondant au plus près des requêtes utilisateurs. Cela permet également de faciliter les requêtes en mode batch, micro-batch ou temps-réel.

Il est donc crucial de ne pas négliger les problématiques d'intégrité des données lors de leur transit d'une source à l'autre.

Problématique d'intégration des données

L'architecture d'un système d'information (ou application complexe) est composée de nombreux logiciels pour la gestion du cache et les recherches full text de données agrégées afin de faciliter leur lecture.

A chaque nouveau besoin, la tendance actuelle est d'ajouter un composant capable de mettre la donnée dans une forme facilitant son usage. Pour la lecture, les architectes ajoutent par exemple des niveaux de cache supplémentaires. Il en résulte un système avec beaucoup de composants interdépendants, ce qui devient très complexe non seulement à comprendre mais aussi à maintenir sur le long terme.



Le fait par ailleurs de garder les données synchronisées entre les différentes représentations et les différents systèmes de stockage devient un vrai challenge. En effet, quand la donnée change dans une source de données, il est nécessaire de s'assurer que la donnée dans les autres sources de données est mise à jour.

Pour pallier cette complexité, les architectures **Microservices** permettent de décomposer un système d'information en des composants indépendants plus orientés métier (et donc implicitement plus petits) afin de faciliter leur maintenance.

Mais faut-il encore que ces services soient indépendants les uns des autres (sinon le problème de maintenance va persister). Aussi, disposer de plusieurs solutions de stockage indépendantes n'est pas un problème en soi. Le problème est que la majorité des solutions de stockage contiennent la même donnée dans des représentations différentes (souvent pour faciliter la lecture). Par exemple, les données dans un moteur de recherche ou dans un cache sont aussi présentes dans une base de données en tant que référentiel (source de vérité).

Techniques de mise à jour de plusieurs sources de données

Pour résoudre les problématiques de mise à jour des données, il existe deux principales techniques :

Le dual write

Cette technique dual-write donne la responsabilité au code applicatif d'écrire les données dans chacune des sources de données appropriées (les différentes bases de données, les solutions de cache, les queues de messages, etc.). Cette approche très populaire, ne fonctionne pas de manière toujours optimale en raison de nombreux problèmes d'accès concurrents comme les « race conditions » et la possibilité d'avoir des échecs partiels. Par exemple, si deux clients essaient d'écrire dans deux sources de données différentes, deux valeurs différentes pour une même clé au même moment, cela peut entraîner des inconsistances, que l'on nomme **"perpetual inconsistency"** entre les deux sources de données.

Avec une majorité de bases de données traditionnelles supportant la spécification XA (2-phase commit), il est possible de lier les écritures à travers une transaction distribuée. Cependant, la majorité des solutions NoSQL ne supportent pas les transactions et encore moins les transactions distribuées. Quand bien même elles les supporteraient, les transactions distribuées ne seraient probablement pas adaptées en raison des problématiques de temps d'exécution liés à la sauvegarde de "locks" intermédiaires qui nécessite un accès disque.

"An append-only persistent log"

Cette technique Append-only consiste à centraliser toutes les écritures en ajoutant l'élément écrit à la fin d'une séquence d'enregistrement. Toutes les écritures sont persistées de manière séquentielle sans concurrence. Cette solution simple, parfois dite « stupide » , est pourtant la solution permettant de garantir toutes les écritures, respectant l'ordre et évitant les « race conditions » .

Les différents types de logs

Les logs jouent un rôle important dans les moteurs de stockage. Ils permettent de résoudre différentes problématiques. Deux grandes familles de logs se distinguent :

a) Write ahead log (**WAL**)
Type particulier de log qui est généralement un "append-only file". Il permet de fiabiliser l'implémentation d'un **B-TREE** (structure de données très courante dans de nombreuses solutions comme dans Redis) en autorisant la modification du B-TREE uniquement.

b) Log-structured storage
Le log est dans ce cas utilisé comme une solution de stockage intermédiaire. C'est le cas dans les solutions **HBase**, **Cassandra** ou **Riak**. Dans ce type de log, on n'ajoute pas uniquement les données dans le même fichier. Le log est alors décomposé en segments (toujours des logs) et à intervalle régulier, le moteur de stockage fusionne les segments et supprime les clés dupliquées.

Le log de réplication

Les architectures Leader-followers utilise un replication-log. Il s'agit soit d'un write-ahead-log (cas de **PostgreSQL**) ou d'un log réplication dédié comme pour la base **MySQL**.

Dans cette topologie, l'ordre des écritures est gardé dans le log de réplication, ce qui permet à chaque **follower** de rejouer les écritures dans le bon ordre, en consommant le log, et ainsi d'assurer que tous les noeuds en réplication obtiennent le même résultat final. A noter qu'en cas d'erreur réseau, le noeud en réplication peut arrêter de répliquer des données, et reprendre dès que le réseau est rétabli.

Change Data Capture (CDC)

Cette technique consiste à répliquer les données d'un système de stockage vers un autre. Pour cela, deux critères sont nécessairement à prendre en compte :

a) La possibilité d'extraire un snapshot de l'ensemble des données à un instant T

b) Un Stream temps réel des changements

Pour certaines compagnies, CDC devient une brique extrêmement importante pour les applications telles que LinkedIn qui a créé **Databus** ou Facebook qui a créé **Wormhole**. Concernant Kafka, depuis la version 0.9, l'API **Kafka Connect** a été introduite afin de connecter Kafka à d'autres systèmes de données comme des bases de données.

Un Connecteur Kafka permet d'utiliser le paradigme CDC afin d'importer un snapshot de données et de mettre en place un flux de données depuis une base de données vers Kafka.

Les indexes secondaires sont très populaires dans le monde des bases de données relationnelles, pour rendre la recherche de données plus rapide sur certains critères.

Un index n'ajoute pas d'informations dans la base de données au sens propre mais représente une structure de données redondante (dans une forme différente) des données d'origine, que l'on peut supprimer sans supprimer les données d'origine.

Cette structure de données d'index est une structure de données de type clé/valeur dont les clés sont le contenu des colonnes que l'on indexe, et les valeurs sont les lignes contenu pour une clé donnée.

Cette création d'index est essentiellement une opération de transformation déterministe, prenant en entrée la table de données et produisant en sortie, un index. Ce processus est fourni par le moteur de base de données. Quand la donnée est mise à jour au niveau de la table, le moteur de base de données met à jour l'index afin qu'il concordent. Techniquement sous le capôt, au moment de la création de l'index, les solutions de bases de données relationnelles parcourent entièrement la table de la base de données. Lors de ce parcours, le contenu de la table peut évoluer. Pour cela, le moteur de base de données a besoin de construire son index depuis un snapshot consistant des données à un instant donné et de suivre la trace de tous les changements. Ce processus est donc similaire au Change Data Capture (CDC).

SIMILARITÉ ENTRE INDEX SECONDAIRES ET CACHE

Un index secondaire est une structure de données redondante qui organise les mêmes données de manière différente afin d'accélérer les requêtes en lecture.

De manière similaire, un cache résulte de l'acquisition d'une donnée (depuis la base de données) et de sa transformation afin d'accélérer les lectures. En d'autres termes, le contenu du cache est dérivé du contenu des bases de données, similaire à un index.

Notion de vues

Il est possible de pré-traiter des données avec des vues :

a) Les vues virtuelles (non-materialized ou virtual view). Il s'agit juste d'un alias d'une requête. Au moment de la lecture depuis la vue, le moteur de base de données transforme la requête demandée en une requête sous-jacente, ce qui n'a pas d'impact sur les performances.

b) Les vues matérialisées (Materialized View) sont très similaires mais possèdent une implémentation radicalement différente. Au moment de la création, la requête a été exécutée et le résultat est écrit sur le disque. On peut considérer cela comme une sorte de cache du résultat d'une requête. Ainsi, comme un cache applicatif et un index secondaire, les vues matérialisées contiennent des données redondées issues de la table source.

Quand les données changent dans la table d'origine, le moteur de base de données est en charge de synchroniser la vue matérialisée. L'avantage de ce mécanisme est d'éviter de laisser la responsabilité d'invalidation du cache à l'application. A l'image d'un index, tout le contenu est pré-calculé avant toutes les demandes d'accès aux données.

Philosophie Unix

La philosophie Unix est un ensemble de principes qui a émergé graduellement durant la conception et l'implémentation des systèmes Unix à la fin des années 60 et durant les années 70. Il y a plusieurs interprétations de la philosophie Unix mais dans la description de Douglas Mcllroy, Elliot Pinson et Berk Tague en 1978, deux caractéristiques fortes se dégagent :

a) Chaque programme doit faire une et unique chose correctement

b) La sortie de chaque programme peut devenir l'entrée d'un autre programme

Il n'est pas rare désormais de considérer Kafka et les Stream Processing comme la réincarnation des Unix pipes au 21eme siècle.

De nombreuses analyses de données peuvent être faites en quelques minutes en utilisant une combinaison de commandes comme awk, sed, grep, sort, uniq et xargs.

Exemple : awk '{print $7}' access.log | sort | uniq -c | sort -rn | head -n

Un programme Unix met en avant la modularité et la composabilité des différents éléments.

Pour permettre la composabilité, tous les outils Unix utilisent une interface uniforme pour les entrées et les sorties: stdin, stout, sterr.

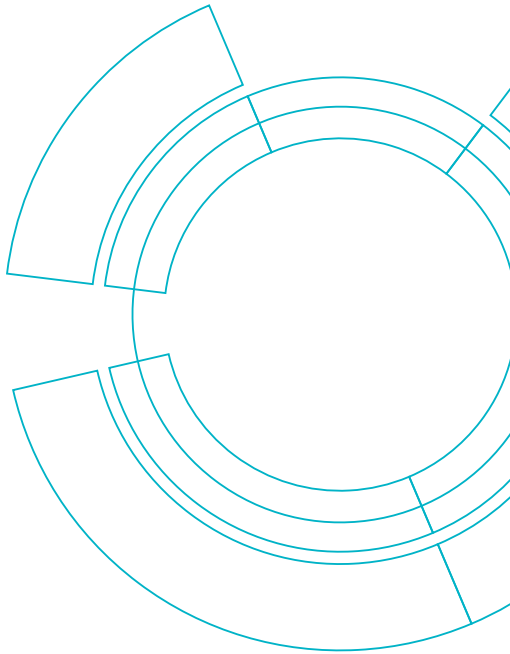
Les différentes solutions de bases de données sont généralement dotées de différents points d'extension comme les procédures stockées (PL/SQL, T-SQL, PL/pgSQL, PL/v8), des types de données personnalisés ou des moteurs de stockage personnalisés. Ces extensions se présentent essentiellement sous forme d'APIs. Il est nécessaire de s'assurer que les applications puissent s'intégrer avec ces APIs. Ces dernières sont conçues pour un usage particulier et ne peuvent pas être utilisés sans risque à d'autres usages. Ils n'ont donc malheureusement pas la même modularité et composabilité qu'Unix.

Un partitionnement avancé

Si Kafka écrivait toutes ses données séquentiellement dans un fichier unique, son début d'écriture serait limité au début d'écriture séquentiel du disque de la machine (aux alentours de 140 Mo par seconde).Pour les usages d'Apache Kafka, ce n'est pas suffisant, les topics de Kafka (représentant les flux de messages) sont découpés en partition. Chaque partition est un log, qui est une séquence ordonnée de messages. Les différentes partitions sont complètement indépendantes les unes des autres et peuvent être localisées sur différents serveurs. Ainsi, Apache Kafka peut scaler horizontalement. Chaque partition est stockée sur disque et répliquée à travers plusieurs machines, permettant de gérer l'échec de machines.

Kafka, un outil central

Kafka est reconnu comme l'un des meilleurs outils pour supporter une forte volumétrie d'activités comme les logs d'un serveur Web ou les activités d'un utilisateur. Les événements sont gardés par le système Kafka pendant une certaine période de temps, puis sont détruits ou persistés dans un autre système de stockage.



A la rescousse des problèmes des pipes Unix

Les pipes Unix possèdent quelques problèmes comme :

- **La limitation à une seule machine**

Kafka est distribué par défaut et tous les connecteurs de traitement de la donnée (stream processors) sont aussi distribués par nature sur plusieurs machines.

- **Communication un à un seulement**

Kafka peut avoir plusieurs producteurs et plusieurs consommateurs. Avoir la possibilité d'avoir plusieurs entrées est important pour des services distribués entre plusieurs machines et avoir plusieurs sorties fait de Kafka un canal de distribution de messages (broadcast channel). C'est la raison d'être de Kafka de permettre à un flux de données d'être consommé indépendamment pour différents usages comme le monitoring ou l'audit, considérés comme en dehors des besoins métier des applicatifs.

- **Pas de tolérance aux pannes**

Kafka fournit une très bonne tolérance aux pannes en permettant à la donnée d'être répliquée à travers plusieurs noeuds (serveurs) Kafka. Ainsi, si un noeud tombe en panne, un autre noeud reprend automatiquement la main.

De plus, si un consommateur tombe en panne et qu'il est redémarré, il peut reprendre sa consommation au dernier point sauvegardé de lecture. Ainsi, il ne manque aucune donnée en entrée.

- **Besoin de « parser » la donnée en entrée et d'espacer la sortie**

Au lieu d'un flux de octets (stream of bytes), Kafka fournit un flux de messages qui est sauvegardé à la première étape.

Cela permet de mettre en avant un format de sérialisation de messages comme JSON, **Avro**, **Thrift** ou **Protocol Buffers**, permettant aux applications en consommation et en production, de travailler en mode programmation Objet et ainsi de supporter l'évolution du schéma de données sans casser la compatibilité.

Traitement des données

L'étape du traitement des données est l'une des étapes les plus stratégiques dans un système Big Data. Trois grandes familles de traitements se distinguent :

- Batch
- Micro-Batch
- Temps-réel (ou Streaming)

Traitement en mode Batch

En mode Batch, l'ensemble des données à un instant T est analysé. D'une durée généralement d'une minute, voire d'une heure, les résultats ne sont disponibles qu'à la fin du traitement. Côté implémentation, il n'est pas rare de retrouver une implémentation MapReduce pour paralléliser les traitements et éventuellement les distribuer sur plusieurs serveurs. Hadoop et de très nombreuses solutions NoSQL proposent une implémentation du modèle de programmation MapReduce et de ses différentes abstractions (**Pig,**

Cascading / **Scaling**), appliquées à des données généralement très volumineuses (> 1 Téraoctet).

A noter qu'en complément, il est aussi possible d'utiliser un moteur de requêtes massivement parallèles (MPP) tel qu'**Impala** ou **Drill**, pour exécuter des requêtes interactives à la manière de SQL.

Traitement en mode Micro-Batch

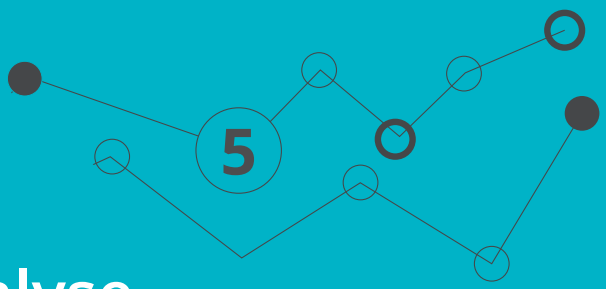
En mode micro-batch, l'ensemble des données est analysé à intervalle régulier, toutes les n secondes. Hérité du mode Batch, ce mode peut effectuer une analyse non pas sur les 6 derniers mois mais sur les 6 dernières secondes. Contrairement au mode batch, cela s'applique généralement à un plus petit volume de données. Côté technologie, le framework le plus populaire est **Spark Streaming** de l'écosystème **Spark**.

Traitement en mode temps-réel

En mode temps-réel, les données sont analysées au fur et à mesure qu'elles arrivent dans le système Big Data. Les traitements s'appliquent alors sur un plus petit volume rendant les résultats disponibles plus rapidement.

De très nombreux outils de gestion de stream existent, avec chacun des niveaux de maturité différents. Les principaux frameworks sont **Apache Storm** ou **Apache Samza** et **Apache Flink**.

A noter que Spark est un framework de cluster computing qui permet de traiter de larges volumes de données en mode distribué. Et le modèle de programmation proposé par Spark est plus simple que celui d'Hadoop et plus rapide en temps d'exécution.



Analyse, Restitution & Visualisation

Une fois stockées et traitées, tout l'enjeu de l'analyse sera de transformer la donnée en connaissance. Et c'est précisément là qu'interviennent les langages de programmation et les plateformes d'analyses.

Il faut préparer les données (identification, nettoyage, formatage pour les rendre prêtes à l'analyse), construire un modèle analytique et tirer des conclusions à partir des connaissances acquises. Pour cela, il est nécessaire de les mettre à disposition via différents canaux comme des APIs ou des outils de reporting.

La datavisualisation, quant à elle, va permettre de restituer les données en les rendant compréhensibles par le métier. N'oublions pas en effet que ce sont les directions métiers qui vont utiliser les informations issues des données. Une communication claire et concise est donc essentielle : les informations doivent être clairement présentées, sous forme de rapport, de graphiques, de chiffres ou encore de recommandations clées.

Valorisation des données

La valorisation des données consiste à générer de la valeur métier. Les techniques issues du Machine Learning permettent d'élaborer et d'implémenter des modèles mathématiques donnant un nouveau sens aux données. On retrouve différentes méthodes :

- **La prédiction :** à partir de données historiques, on prédit ce qui est susceptible de se produire dans le futur.
- **La détection d'anomalies** consistant à repérer/identifier des données qui sortent de l'ordinaire.
- **La segmentation** permettant de séparer les données en groupes de données, ayant des caractéristiques communes.

Restitution

La restitution des données consiste à fournir les données aux utilisateurs du système Big Data, de manière brute ou via des APIs.

Au-delà de ces APIs, dans un écosystème favorisant la visualisation, il est parfois plus simple de mettre à disposition des outils graphiques.

C'est alors que de nombreux éditeurs de la BI ont fait évoluer leurs produits afin de pouvoir gérer en plus de leurs sources de données « classiques », des sources de données d'un système Big Data. C'est le cas par exemple de **Tableau** ou **Pentaho**.

Côté solutions NoSQL, on choisira généralement des solutions comme **MongoDB** ou CouchBase pour parcourir les résultats de requêtes complexes. Lorsque les requêtes sont rapides, un moteur d'indexation comme **ElasticSearch** ou **SolR** pourra être utilisé. Pour la visualisation des résultats, **ElasticSearch** peut être enrichi d'un dashboard **Kibana** offrant une bonne base.

Mais dans tous les cas, lorsque les besoins de restitution des données sont complexes et reposent sur un grand nombre de critères, il est indispensable de concevoir et développer une application Web de visualisation dédiée. Cette application pourra, le cas échéant, s'appuyer sur une librairie JavaScript Open Source, **D3.js** qui facilite la manipulation et l'affichage des données sous différentes représentations graphiques.

D3.js est très populaire chez les développeurs pour créer des graphes tout comme **Nvd3**. Il y a également **Google Charts** qui est un moyen rapide de faire des graphes.

GESTION DES GRAPHES

Côté framework **Spark**, la gestion des graphes s'appuie sur **GraphX**, un framework distribué de graphes simples (parfois au profit de la performance) qui s'appuie sur une API de modélisation et d'exécution.

Côté **Flink**, le projet **Gelly** permet de gérer des graphes en offrant des utilitaires d'analyse, de traitement ainsi que des algorithmes dédiés aux graphes.

6

Architectures des systèmes Big Data

Pour un système Big Data, la solution idéale est de définir une architecture flexible et adaptée aux différents cas d'utilisation. Il n'existe cependant pas de modèles d'architecture parfaits ou pouvant s'appliquer partout avec pertinence. Avant de démarrer votre projet Big Data, il est donc préférable d'identifier les propriétés communes et de connaître le fonctionnement des différentes architectures.

Des propriétés communes

Parmi les grandes caractéristiques que l'on attend d'une Architecture Big Data, on retrouve généralement :

- Robustesse et tolérance aux pannes
- Faible latence en lecture et en écriture (ou mise à jour)
- Scalabilité
- Généralisation
- Extensibilité
- Capacité à pouvoir exécuter des requêtes arbitraires
- Maintenance minimale
- Débuggabilité

Robustesse et tolérance aux pannes

Les architectures Big Data par définition sont des architectures distribuées. Le système doit alors continuer à se comporter correctement malgré l'indisponibilité de certaines parties du système comme l'arrêt de serveurs. Un autre élément souvent oublié est la tolérance aux erreurs humaines, en particulier dans les requêtes de mise à jour des données.

Faible latence en lecture et en écriture (ou mise à jour)

La majorité des applications d'un système Big Data requièrent une faible latence en lecture pouvant aller parfois jusqu'à la microseconde. En revanche, côté écriture et mise à jour, les besoins sont souvent très différents entre les applications. Certaines nécessitent une propagation immédiate des modifications entre les serveurs, quand d'autres acceptent selon le métier une latence pouvant atteindre l'heure.

Scalabilité

La scalabilité est la capacité à maintenir la performance face à l'augmentation des données ou de la charge, en ajoutant des ressources comme des serveurs. Au regard des besoins métier fluctuants comme l'augmentation du volume de ventes pour un site e-commerce, cette propriété est aujourd'hui de plus en plus indispensable dans les systèmes Big Data.

Généralisation

La mise en place d'une architecture Big Data est souvent coûteuse. La généralisation est la capacité à étendre un système existant à un maximum de cas d'usage. Cette capacité peut être atteinte dans les architectures Lambda qui consistent à appliquer une fonction de transformation sur un jeu de données source, permettant d'aboutir à un second jeu de données dérivé du premier, et répondant de ce fait, à des cas d'utilisation différents.

Extensibilité

Son principe est d'avoir la possibilité d'ajouter une fonctionnalité avec un coût d'implémentation minimale.

Capacité à pouvoir exécuter des requêtes arbitraires

La capacité d'un système Big Data à exécuter des requêtes particulières (requêtes "ad hoc queries") est extrêmement importante. Sa capacité à extraire un ensemble de données de manière arbitraire offre de nouvelles opportunités en termes d'optimisation métier et de nouveaux cas d'usages.

Maintenance minimale

La maintenance d'un système Big Data est la capacité à maintenir un système le plus « lisse » possible. Techniquement, cette maintenance minimale implique une anticipation dans l'ajout de machines pour scaler et garder la plateforme Big Data toujours active. En termes d'implémentation, cette maintenance nécessite de choisir des composants ayant des implémentations minimales.

Débuggabilité

Un système Big Data doit fournir toutes les informations nécessaires pour identifier les dysfonctionnements des composants du système au regard de leur comportement attendu. L'idée est d'auditer une valeur donnée dans le système, et toutes les étapes du processus ayant généré cette valeur. Cette propriété "Debuggability" est en particulier atteinte à travers les architectures Lambda.

Disposer de toutes les propriétés ci-dessus est extrêmement difficile pour un système Big Data. Une majorité de ces propriétés émergent naturellement à travers une bonne conception d'architecture Big Data.

FAUT-IL PRENDRE EN COMPTE TOUTES LES CARACTÉRISTIQUES D'UNE ARCHITECTURE ?

L'architecture est avant tout une activité de prise de décision selon un contexte donné (fonctionnel, technique, organisationnel, etc.) et il est particulièrement complexe de réunir pour une même architecture, l'ensemble des propriétés énoncées ci-dessus.

Certaines caractéristiques n'auront en effet pas le même niveau d'importance, et il faudra donc privilégier la pondération sur les critères les plus indispensables, en faisant appel à un expert lors de la phase de conception.

Architecture Hadoop

Aucune technologie ne permet de résoudre tous les types de problèmes posés. Hadoop a posé les bases du Big Data (surtout en termes de traitement). La plateforme est capable de traiter des volumes importants de données, mais avec une latence importante en raison d'un traitement en mode batch.

Dans ce type d'architecture Hadoop, les données sont ingérées dans **HDFS** (Hadoop Distributed File System), **AWS S3**, les bases de données **SQL** et **NOSQL** ou les moteurs de recherche. Ces données sont injectées via des services tels que **Flume** pour les agrégations de logs ou Scoop pour la partie interopérabilité.

Ensuite des jobs d'analyse sous forme Hadoop MapReduce, Spark ou d'autres outils sont utilisés. Les architectures Hadoop amènent de nouveaux challenges comme la surveillance des temps d'exécution, des jobs et de l'ordonnancement des jobs Hadoop.

L'OUTIL SQOOP DE L'ÉCOSYSTÈME HADOOP

Projet sous la fondation Apache, Sqoop a pour objectif de faciliter la cohabitation des systèmes traditionnels comme les bases de données relationnelles avec la plateforme Hadoop à travers des connecteurs d'import et d'export. Il est ainsi possible d'exporter des données depuis un SGBDR pour pouvoir exécuter des traitements sur ces données via un MapReduce Hadoop, ou stocker le résultat d'un traitement Hadoop dans un SGBDR. Devant l'intérêt et la popularité de l'outil, certains éditeurs comme DataStax (éditeur de la solution Cassandra) fournissent désormais un connecteur Sqoop pour leur base de données NoSQL .

Architecture dite "Batch incrémental"

Les traitements incrémentaux vont différer des traitements batch normaux par la possibilité de ne traiter que les nouvelles entrées, tout en proposant un résultat qui tienne compte de l'ensemble des données. L'agrégation des traitements incrémentaux est prise en charge par les frameworks technologiques.

Architecture Lambda

Imaginées par Nathan Martz et James Warren, les architectures Lambda permettent de construire une vision complète en valorisant les données. Ce type d'architecture adresse des problématiques complexes via un modèle hybride en mélangeant des traitements temps réel et des traitements batch, indépendamment du fonctionnement des différentes technologies.

Ces architectures Lambda sont constituées généralement de trois couches :

- **Couche Batch**
Cette couche Batch gère le stockage de l'ensemble des données et doit assurer leur intégrité. D'un point de vue technologique, on retrouve les solutions Hadoop et les bases de données relationnelles.

- **Couche de service**
La couche de service (ou "Serving Layer") permet de stocker les vues créées par la couche Batch. On y trouve généralement des solutions NoSQL comme Cassandra ou des bases de données orientées temps comme Druid.

- **Couche temp réel**
La couche Temps réel (ou "Speed Layer") permet de traiter les données récentes. Généralement optionnelle, cette couche a un objectif principalement d'optimisation, avec généralement des solutions NoSQL en mémoire telles que Redis. • • •

L'intérêt de ces architectures Lambda est de conserver les données brutes afin de pouvoir les traiter de nouveau, et ainsi de fournir la vision la plus récente possible au métier.

Cependant les architectures **Lambda** sont en déclin en raison des moteurs en temps réel (comme **Apache Kafka Stream**, **Apache Apek**, **Apache Samza** ou **Spark Streaming**) capables d'exécuter les requêtes directement sur les flux continus d'événements sans avoir stocké les données.

Les architectures Lambda permettent de disposer de la propriété "Debuggability" à travers la nature « fonctionnelle » de la couche "Batch" et la capacité à exploiter des algorithmes de re-calculs.

POINTS D'ATTENTION D'UNE ARCHITECTURE

Une architecture d'un système Big Data amène une certaine complexité dans le système d'information (SI). Parmi les points d'attention :

- **La multiplication de solutions technologiques**, en particulier dans la couche Batch et la couche Temps Réel. Chaque solution est un produit en soi nécessitant un apprentissage approfondi et une maintenance pour fonctionner correctement.
- **Éventuellement la dispersion de la logique métier**. Il est très souvent observé dans des architectures Lambda, la duplication du code métier présent sur la couche Batch et sur la couche Temps réel, rendant ainsi la maintenance plus fastidieuse.

Les architectures émergentes à base d'un Stream d'événements

L'avantage d'un stream d'événements est d'avoir plusieurs consommateurs du même événement de la donnée. Grâce à cela, il est possible d'avoir un consommateur qui a pour simple rôle de stocker les événements et un autre consommateur qui crée et stocke l'agrégation des données.

Stockage d'événements

Le stockage d'événements consiste à écrire des éléments dans un log. Dans l'exemple de l'analyse du trafic d'un site Web, l'administrateur est rarement intéressé par tout l'historique des pages vues, mais par des données agrégées comme le nombre de visiteurs uniques.

Depuis les flux d'événements, qui sont la source de vérité, on génère des agrégats de données dédiés à la lecture uniquement, qui sont stockés dans des solutions logicielles de cache afin de réduire le temps d'accès à la donnée.

Ces agrégats permettent de représenter la donnée dans une forme répondant au mieux aux requêtes des utilisateurs.

STREAM PROCESSING

L'émergence de la dénormalisation

Pour optimiser la lecture, il est possible de dénormaliser la donnée, en la dupliquant à plusieurs sources afin que la lecture soit plus rapide.

La résilience

La capacité de résilience du Stream Processing est particulièrement intéressante et permet de proposer différents patterns :

- Le message est pris plus d'une fois
- Le message est pris au moins une fois
- Le message est pris exactement une fois

L'UTILISATION DE REDIS POUR ATTEINDRE DE TRÈS BONNES PERFORMANCES EN LECTURE

Le premier intérêt de Redis est d'être une base de données en mémoire permettant d'optimiser les temps d'accès à la donnée par rapport à un accès disque (500 fois plus rapide qu'un accès SSD en moyenne).

L'autre principal intérêt de Redis est sa capacité à proposer des structures de données riches permettant de travailler au plus près de la nature des données. Contrairement aux autres solutions de données, les valeurs dans Redis ne sont pas considérées uniquement comme un blob. A contrario, Redis propose des structures de données riches et des APIs associées pour accéder plus simplement aux données comme des grappes d'objets sans coût de sérialisation / désérialisation, des commandes dont la complexité du temps d'exécution est en O(1).

De plus, Redis se prête parfaitement à une solution de cache en fournissant d'une part, de nombreuses politiques d'éviction et en permettant d'autre part, de mettre à jour son cache ou de faire de l'éviction de données proactive simplement. Ainsi dupliquer, les données dans Redis est parfait pour atteindre des lectures rapides.

L'event Sourcing au service des bases de données

L'utilisation d'une approche Event Sourcing pour l'usage de nos bases de données est un avantage certain par rapport aux usages traditionnels, en particulier pour la mise à jour et la suppression de la donnée. Plus précisément, cette approche apporte un couplage lâche entre le code qui écrit la donnée et celui qui la lit.

De plus, elle permet une meilleure gestion des erreurs car à tout moment, il est possible de rejouer l'historique de toutes les modifications effectuées sur la base de données afin d'obtenir l'état du résultat final.

DE HAUTES PERFORMANCES EN LECTURE ET EN ÉCRITURE

Le débat concernant la normalisation (favorisant les performances en écriture) versus la dénormalisation (favorisant les performances en lecture) existe uniquement en raison de l'hypothèse que la lecture et l'écriture de la donnée utilisent le même schéma. Si on sépare la modélisation des lectures, de la modélisation d'écriture, il est alors tout à fait possible d'avoir à la fois une lecture et une écriture rapide. C'est l'approche que propose CQRS (Command Query Responsibility Segregation).

Implémentation d'un flux

Un stream doit être implémenté comme un "log", c'est-à-dire comme une séquence d'événements en mode "append-only" dans un ordre déterminé.

JMS et AMQP non adaptés pour les Stream

L'ordre des événements est extrêmement important. L'API JMS (Java Messaging Service) ou le protocole **AMQP** (Advanced Messaging Query Protocol) avec les systèmes de messaging qui les implémentent comme **ActiveMQ Artemis** ou **RabiitMQ** ne garantissent pas cet ordre.

Kafka, un log visible à tout le monde

Au lieu de considérer le log comme un détail d'implémentation, **Kafka** ne cache pas son log des données car l'outil l'expose à tous ses consommateurs permettant de créer de multiples applications (potentiellement très différentes). Le log devient ainsi une API publique.

Architecture Kappa

Imaginées par Jay Kreps, les architectures Kappa sont nées en réaction à la complexité des architectures Lambda pour fournir une donnée plus à jour. Avec ces architectures Kappa, les couches batch et temps réel fusionnent. Côté technologique, les principaux frameworks tels que **Spark** et **Flink** permettent d'adresser les deux modes (batch et temps réel). Une autre caractéristique des architectures Kappa concerne le stockage : celui-ci est restreint à un système de fichiers de type log-append. On retrouve généralement un système de message de type publish-subscribe comme Apache **Kafka**.

Architecture SMACK

Les architectures SMACK pour **Spark-Mesos, Akka-Cassandra-Kafka** sont différentes des architectures Lambda et des architectures Kappa car elles sont composées d'une suite de technologies considérées comme matures dans l'écosystème Big Data.

Une des idées avec les architectures SMACK est donc de traiter les données à un coût plus faible.

Spark est utilisé pour le traitement batch, couplé avec la brique **Spark Streaming** pour la partie temps réel.

AKKA, UN FRAMEWORK ACTEUR TENDANCE

Inspiré de Erlang proposant une Architecture hautement concurrente et basée sur les événements, Akka est un framework sur la JVM, destiné à construire des applications concurrentes distribuées et résilientes, sur le modèle de programmation Acteur à base de messages. Ce paradigme sépare l'émetteur du message lui-même via une communication asynchrone.

Infrastructure Big Data

Concevoir un système Big Data renvoie à des problématiques telles que le dimensionnement, la sécurité ou même la gouvernance des données. Une fois en production, de nombreux outils de supervision sont nécessaires pour s'assurer du bon fonctionnement du système et remonter, les cas échéant, différentes alertes en cas de disfonctionnement.

L'ÉMERGENCE DE LA CONTAINERISATION

Un environnement Big Data est composé de l'agrégation de plusieurs technologies, de configurations spécifiques et d'architectures techniques (généralement distribuées) plus ou moins complexes. Pour tester son système Big Data, **l'approche Container comme Docker** permet d'aider à simuler les différents environnements et les différentes configurations, facilitant ainsi la mise en place d'un système Big Data.

Sécurité

La sécurité est donc essentielle dans le domaine du Big Data et se décline selon 4 axes :

- Périmètre
- Accès
- Donnée
- Visibilité

Sécuriser l'accès au Cluster revient à mettre en place un système d'authentification sur le Cluster. Sécuriser l'accès aux données correspond à la mise en place d'un système d'autorisations et de permissions sur les données. Une fois que l'accès aux données est sécurisé, ce sont les données elles-mêmes qui doivent être sécurisées.

Concernant les aspects de sécurité pour la plateforme Hadoop, il existe des solutions comme **Sentry** pour l'identité et les ACL se présentant sous la forme d'un plugin et d'un serveur. Le plugin peut s'intégrer à « toutes » les briques de l'écosystème **Hadoop**.

Enfin, il existe aussi **Knox** qui est une Gateway exposant des API REST, permettant d'interagir avec un cluster Hadoop (et d'autres services comme **Hive**, **Impala**, etc.).

Dans ce cas là, on se retrouve avec tout ce qui est lié au HTTP (SSO, OAuth, etc.) pour gérer l'identité. Il est également possible de gérer l'autorisation (ACL) avec **Knox**.

Cloud

Il existe de très nombreuses offres Cloud. La majorité de ces offres permet de se concentrer sur le métier, pour faciliter l'adoption du Big Data, avec la promesse d'une réduction des coûts par une facturation à l'usage.

Les principaux acteurs fournissant des offres relativement complètes et matures sont Google, Rackspace, Amazon et Microsoft.

Dimensionnement

Un système Big Data ne signifie pas pour autant mettre en place une infrastructure surdimensionnée. Il sera au contraire préférable de choisir des serveurs dit « normaux », couplés à un mécanisme de scalabilité horizontale. Cependant, en fonction de chaque outil, il est nécessaire d'examiner en détail les prérequis RAM, CPU, Disque.

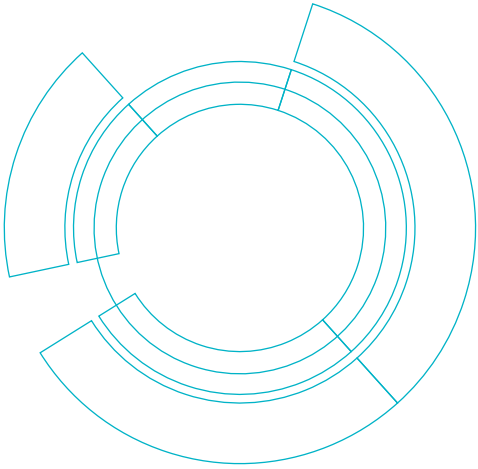
Supervision

Tout système, dès lors qu'il est exploité en production, nécessite une surveillance permanente (ou supervision), pour garantir sa disponibilité. Les besoins les plus basiques en matière de supervision sont de :

- remonter les alertes en cas de panne pour intervenir au plus tôt.
- mettre à disposition un maximum d'informations aidant aux investigations visant à une action corrective (logs, tableaux de bord etc.).
- permettre une reprise sur panne le plus rapidement possible.
- disposer d'indicateurs techniques (système, réseau etc.) permettant d'identifier les causes de dysfonctionnements.
- avoir un suivi de son utilisation et d'indicateurs fonctionnels, permettant d'anticiper la saturation d'un système, et les besoins à venir.

En outre, à cela s'ajoutent les besoins d'administration :

- arrêt / relance,
- mise à niveau de composants,
- changement de configuration,
- etc.



Ces besoins sont d'autant plus justifiés sur un système distribué, et ce, pour les raisons suivantes. Tout d'abord, du fait de leur caractère distribué, les solutions Big Data sont davantage exposées aux pannes :

- Un cluster repose sur une couche réseau et dépend donc de son fonctionnement (problèmes de coupures réseau, latence, débit etc.)
- Un cluster, même s'il est vu comme un tout par son utilisateur, dispose de ressources morcelées : dans certains cas, il peut donc suffire d'un dysfonctionnement sur une seule machine pour créer un goulot d'étranglement pénalisant le cluster dans son ensemble !
- Certains écosystèmes comme Hadoop souffrent encore de leur jeunesse, et sont donc sujet à de nombreux bugs.

De plus, les investigations suite à des incidents sont plus difficiles, car :

- Retrouver sur quelle machine survient une panne n'est pas chose aisée, sans aide à la recherche.
- Un système distribué met en jeu des algorithmes plus complexes, les pannes sont donc plus difficiles à comprendre et à résoudre.

Enfin, les tâches administratives, qui peuvent s'avérer triviales sur un seul serveur, prennent une toute autre envergure lorsqu'il s'agit de les mettre en oeuvre sur un cluster de 200 machines ! Imaginez un instant par exemple l'ampleur que peut prendre une simple mise à niveau du JDK sur votre cluster… C'est là que l'on comprend l'utilité du DevOps dans le cadre du Big Data.

Que faut-il superviser ?

Nous venons de le voir, superviser la plateforme est indispensable. Mais quels seront exactement les besoins d'un système distribué ?

Il faut tout d'abord bien sûr connaître le "Healthcare" des machines :

- identifier les pannes machines, et remonter les alertes.
- avoir des métriques sur l'utilisation des ressources (CPU, mémoire, disque, I/O...), afin d'identifier les goulots d'étranglement au sein d'une machine.

Puis, une vision globale du cluster pourra également aider à optimiser les ressources et les coûts :

- quelle utilisation est faite du cluster ? Est-il sous-utilisé, ou au contraire en permanence saturé ? Quelle est la répartition dans le temps de son utilisation ? Y a-t-il des pics d'utilisation ? (les réponses à ces questions pourraient orienter vers une stratégie Cloud « élastique »).

- Y-a-t-il des goulots d'étranglement au niveau du réseau identifié, qui pourraient amener à changer sa configuration ? (noeuds d'orchestration, noeuds "worker"...).

Par ailleurs, il faut aussi envisager la supervision des traitements qui tournent, ou qui ont tourné sur le cluster :

- les jobs se sont-ils terminés correctement, et si oui, en combien de temps ? Comment ont-ils été découpés ?

- quels sont les jobs en erreur et pourquoi ?
  Y a-t-il eu une reprise sur panne automatique ? Faut-il remonter une alerte ?

Il faut également envisager des indicateurs de performance sur l'exécution des jobs pour identifier les goulots d'étranglement et les points d'optimisation possibles d'APIs Rest comme :

- les statistiques sur les temps de transactions
- les statistiques sur les volumes de transactions

Enfin, en termes d'administration, les besoins vont des plus basiques (start / stop des noeuds) au plus complexes :

- configuration en masse des noeuds.
- provisionning (ajout/suppression de machines au sein du cluster).
- déploiement ou mise à niveau de composants Hadoop sur chaque machine du cluster.

Les outils de supervision ?

Chaque solution vient avec un ensemble d'outils ou d'utilitaires de supervision permettant de superviser cluster, node, mais aussi buckets.

Cloudera dispose du Cloudera Manager. MapR dispose de sa console d'administration MapR Control System. Mais il existe aussi des solutions Open-source, telles qu'Apache Ambari, qui font partie de la distribution Hortonworks.
Côté Cassandra, le Cassandra Ops Center rendra ce service. Couchbase quant à lui, dispose d'un ensemble d'API Rest permettant d'effectuer le monitoring sur un cluster de nœuds.

MongoDB dispose d'une suite d'utilitaires tels que mongoStat, MongoTop, et tout comme Couchbase, d'un ensemble de commandes de suivi de l'exécution.

CONCLUSION

Jusqu'à encore récemment, les systèmes Big Data étaient orientées en mode "Batch". Les données étaient capturées au sein d'un système distribué ou d'un cluster de base de données. Désormais, les systèmes Big Data évoluent pour être orientés en flux continu de données (Stream) afin de traiter la donnée dès qu'elle arrive. Les nouveaux systèmes Big Data sont précisément là pour injecter et traiter en continu des données voire des flux de données à l'infini. Cependant, les systèmes à base de flux sont plus complexes à construire.

Afin de réaliser une transition en douceur, ces nouveaux systèmes Big Data supportent à la fois un mode Batch et des traitements interactifs. Le flux continu des données est un élément indispensable à la pertinence des analyses de tendance et à la précision d'une prédiction. Pour affiner ces prédictions, il est possible de mettre en place du Machine Learning tout en continuant à collecter les données en temps réel, grâce à des architectures adaptées. C'est ainsi que le Big Data va accompagner la révolution des APIs et leur ouverture.

Vision et prospective

Prochaine étape : anticiper les promesses du Big Data

Quel avenir le Big Data réserve-t-il à l'entreprise ? Alors que la maturité augmente dans les organisations sur la nouvelle place que prend la donnée au sein de leur business, les réponses à cette question se font plus précises.

Ce livre blanc a pris le parti de s'ancrer dans le concret, en abordant aussi bien les stratégies et orientations métier ou organisationnelles que doivent retenir les dirigeants, que les aspects plus techniques du Big Data. Nous avons voulu montrer comment « activer » le Big Data dès aujourd'hui ; étape par étape et choix après choix.

Il n'est cependant pas inutile dans cette optique, de se projeter vers ce qui attend les décideurs dans quelques mois ou années. Les choix qu'ils ont pu faire ou qu'ils feront (nous l'espérons, en partie grâce à ce document) les engageront pour les futures transformations de leur entreprise.

Des secteurs à suivre

Au demeurant, la question de l'avenir du Big Data mérite d'apporter deux éclairages complémentaires : l'un sur les possibilités ouvertes par ces technologies, secteur par secteur, en termes de nouveaux usages et de nouveaux modèles ; l'autre sur l'évolution des « façons de faire » du Big Data en tant que telle dans les entreprises.

Bien sûr, ces quelques lignes n'ont pas la prétention de l'exhaustivité sur l'un ou l'autre de ces sujets. Il existe d'ailleurs aujourd'hui une littérature abondante sur ces aspects, qui viendra compléter adroitement la lecture de ce livre blanc. Pour autant, donner un aperçu de ce qui

nous semble en 2017 être les évolutions les plus représentatives du « nouvel univers de la donnée » aidera sans nul doute les acteurs à mettre en perspective les conseils que nous avons présentés dans ces pages.

Ainsi, du point de vue des changements majeurs que nos sociétés sont sur le point de connaître, il est clair que la santé, en particulier au travers de la médecine, se dessine comme l'une des disciplines les plus prometteuses en termes de Big Data. A l'avenir, les hôpitaux seront en effet en mesure de mieux faire fonctionner leurs différents services, de mieux distribuer les médicaments, d'optimiser les procédures et d'anticiper la disponibilité des places.

L'analyse à grande échelle, à partir des données de milliers de patients, va permettre une nouvelle classification des pathologies, notamment dans les cas de cancer, et la mise en place de thérapies mieux ciblées. La prescription de médicaments, en prenant en compte le patrimoine génétique du patient, sera elle-même plus adaptée, mieux dosée et permettra des cures plus ciblées. Ce sera l'avènement d'une médecine personnalisée. Enfin, avec l'Open Data, les analyses pourront s'effectuer sur des échantillons beaucoup plus larges, et le croisement de ces données médicales, rendues anonymes, avec d'autres données économiques, sociales ou environnementales offrira de nombreuses avancées pour la recherche. A titre d'exemple, les données médicales de patients sur une zone donnée, croisées au taux de pollution de cette même zone, pourront intéresser des décideurs politiques, des travailleurs sociaux ou même des chercheurs. Les innovations et les réalisations qui vont être mises en place dans les années à venir dans le secteur de la santé vont donc être source d'inspiration et procureront des enseignements majeurs à tous les autres acteurs de l'économie.

L'industrie sera également à suivre avec attention. Le Big Data est d'ores et déjà en train de jouer un rôle déterminant en termes d'efficacité opérationnelle notamment dans la détection de pièces mécaniques défectueuses, résultant de l'analyse de données en provenance des capteurs installés sur les machines de fabrication. Les manufactures peuvent désormais anticiper l'obsolescence de certaines pièces et ainsi éviter l'arrêt soudain de leurs lignes de production, en raison de défaillances inattendues.

Dans l'industrie comme dans la santé, où les environnements et les règles sont fortement contraints, chaque « réussite Big Data » ouvrira un peu plus la voie à des avancées sérieuses pour d'autres entreprises. Ces dernières pourront ainsi accéder plus facilement aux recettes d'industrialisation, d'exploitation à grande échelle, de sécurité et de vision profondément métier, de ces projets, pour les traduire chez elles.

De nouvelles interrogations

Ces inspirations ne seront toutefois exploitables qu'à condition d'anticiper un certain nombre de questions supplémentaires. La capacité à mettre en place un projet Big Data, de sa définition stratégique à sa mise en œuvre technique, n'est en quelque sorte qu'une première pierre. En s'adaptant à la nouvelle place de la data dans son univers professionnel, l'entreprise sera amenée à intégrer l'ensemble de ses réflexions Big Data plus profondément dans son fonctionnement quotidien.

Elle sera amenée à aller plus loin : comment accélérer la réalisation des futurs projets, pour pouvoir suivre la course effrénée de l'innovation ? Quelles compétences faudra-t-il absolument garder en interne et quelles sont celles qui pourront être externalisées pour gagner en agilité et en capacité d'adaptation face aux nouvelles technologies Big Data ? Sera-t-il possible de monétiser directement ce Big Data, vu par le prisme de l'entreprise ? Et au-delà de l'amélioration des services, quels sont les nouveaux marchés que cette maturité croissante sur le Big Data va permettre d'attaquer ?

Quels nouveaux business modèles pourront répondre à toutes ces questions et permettront à l'entreprise d'exister encore dans 20 ans ?

Nous savons tous que le nombre et l'intérêt potentiel des données à disposition des entreprises ne cessera d'augmenter dans les années à venir. Les avancées en termes d'intelligence artificielle, au sens large, vont ouvrir également de plus en plus de portes en termes d'usages et d'automatisation des processus. Ces deux aspects seront également intégrés de manière de plus en plus transparente dans les outils personnels et professionnels du quotidien. Les changements qui s'annoncent seront donc aussi profonds que pérennes.

Reste que les organisations qui auront pu s'emparer plus naturellement et intuitivement de cette montée en puissance tireront une valeur sans commune mesure avec celles qui la subiront et la digéreront difficilement après une adoption au forceps. Et dans ce cadre, l'ouverture d'esprit des dirigeants et de leurs équipes sera un facteur clé pour anticiper les prochaines étapes.

Auteurs

Livre blanc Big Data : Edition 2017

Patrick Allain

Jean Baylon

Grégory Boissinot

Bruno Doolaeghe

Cécile Hui-Bon-Hoa

Bertrand Lehurt

Nathanaël Marchand

Martine Metzger-Douçot

David Wursteisen

Mathilde Richard

www.soat.fr - blog.soat.fr

Sequana 1 - 89 quai Panhard et Levassor - 75013 Paris

01 44 75 42 55

Conception graphique et réalisation : **MAP Advertising**